

Exploring the Integration of Wavelet Transforms with U-Net for Polyp Segmentation

James Medley

MComp(Hons) Computer Science and Artificial Intelligence
The University of Bath
2024-2025

Exploring the Integration of Wavelet Transforms with U-Net for Polyp Segmentation

Submitted by: James Medley

Copyright

Attention is drawn to the fact that copyright of this dissertation rests with its author. The Intellectual Property Rights of the products produced as part of the project belong to the author unless otherwise specified below, in accordance with the University of Bath's policy on intellectual property (see https://www.bath.ac.uk/publications/university-ordinances/attachments/Ordinances_1_October_2020.pdf).

This copy of the dissertation has been supplied on condition that anyone who consults it is understood to recognise that its copyright rests with its author and that no quotation from the dissertation and no information derived from it may be published without the prior written consent of the author.

Declaration

This dissertation is submitted to the University of Bath in accordance with the requirements of the degree of Bachelor of Science in the Department of Computer Science. No portion of the work in this dissertation has been submitted in support of an application for any other degree or qualification of this or any other university or institution of learning. Except where specifically acknowledged, it is the work of the author.

Abstract

This project investigated the integration of wavelet transforms into the U-Net architecture to enhance semantic segmentation in medical imaging. Two core approaches were explored: replacing U-Net encoder layers with a fixed wavelet scattering network, and substituting conventional downsampling and upsampling operations with real and complex-valued wavelet transforms. A series of hybrid scattering models (WSN-UNet-J1–J4) were proposed and evaluated to assess how the depth of encoder replacement affects segmentation performance and data efficiency, particularly in low-data regimes. Additionally, wavelet-enhanced models (LSW-iDWT and LSW-iDTCWT) were developed to improve the reconstruction of fine spatial detail through wavelet-based upsampling in the decoder. The complex DTCWT-based model outperformed established baselines including U-Net++ and DUCK-Net on the Kvasir-SEG dataset, demonstrating the potential of integrating signal processing techniques into deep learning architectures to improve both interpretability and segmentation performance.

Contents

1	Introduction	1
2	Literature and Technology Survey	3
2.1	CNNs for Image Segmentation	3
2.2	The Evolution of Segmentation Architectures	4
2.3	Recent Novel Approaches in Segmentation	6
2.4	Applications of Wavelets in Deep Learning	8
2.4.1	Convolution with the Fourier Transform	8
2.4.2	Wavelet Transforms	8
2.4.3	Wavelet Enhanced U-Net for Segmentation	13
2.5	Wavelet Scattering Networks	15
2.5.1	Theoretical Foundation of Wavelet Scattering Networks	15
2.5.2	Deep Hybrid Networks: Combining Wavelet Scattering Networks with CNNs	17
2.6	Segmentation Datasets	18
2.6.1	Kvasir-SEG	18
2.6.2	DRIVE: Retinal Blood Vessel Segmentation	19
3	Methodology	20
3.1	Research Questions	20
3.2	Experiment Design	21
3.2.1	Proposed Model Architectures	21
3.2.2	Dataset Preparation	24
3.2.3	Bayesian Hyperparameter Optimisation	25
3.2.4	Training Procedure	30
3.2.5	Evaluation Strategy and Performance Metrics	32
4	Results and Discussion	33
4.1	Hybrid Scattering Models	33
4.1.1	WSN-UNet-J1	33
4.1.2	WSN-UNet-J2	34
4.1.3	WSN-UNet-J3	35
4.1.4	WSN-UNet-J4	36
4.1.5	Discussion	37
4.2	Wavelet Enhanced U-Nets	38
4.3	Overall Discussion	40
5	Conclusions	43

Bibliography	45
A Optimal Hyperparameters	49
B Bayesian Credible Intervals of Metrics	50
C Enhancing Decoder Input with Raw RGB Features	51
D Evaluating the Role of Intermediate Skip Connections	52
E Data Preparation and Training Procedure Differences for DRIVE	53
F WaveletUp Module: Inverse DTCWT Decoder Block	54
G Code Repository and Datasets	56

List of Figures

1.1	Semantic image segmentation, adapted from Chen et al. (2019)	1
2.1	Abstracted illustration of a convolutional layer followed by a max-pooling operation from Li et al. (2021)	3
2.2	The U-Net architecture diagram, from Ronneberger, Fischer and Brox (2015). As described in the original: Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.	4
2.3	ResNet shortcut connection over block. Taken from He et al. (2016)	5
2.4	The DUCK-Net architecture diagram, from Dumitru, Peteleaza and Craciun (2023)	7
2.5	Convolution with the Fourier Transform. The input and Sobel kernel are transformed to the frequency domain, multiplied pointwise, and transformed back with the inverse FFT.	9
2.6	Comparison of time-domain signals and corresponding continuous wavelet transforms (using the complex Morlet wavelet) for a single chirp (left) and a composite chirp signal (right). The single chirp increases linearly from 5-50 Hz, while the composite signal includes an additional chirp from 20-100 Hz. Note how the CWT distinctly separates the two superimposed signals in the time-frequency domain.	10
2.7	Heisenberg uncertainty boxes from Mallat (1999). Each box illustrates the time-frequency localisation of a wavelet at a given scale. Smaller scaled wavelets result in finer time resolution, but coarser the frequency resolution. The opposite is true for larger scales.	10
2.8	The discrete wavelet transform ($J=1$) applied to a retinal image. The left image shows the input, and the right is an arrangement of the four labelled coefficients. Both the left and right images are the same resolution, demonstrating the non-redundancy of the transform.	12
2.9	Pyramid Architecture for the 2-D DWT, from Mallat (1989)	12
2.10	2-D impulse responses of DTCWT, and impulse responses from DWT. Figure taken from Kingsbury (1998).	13
2.11	The DTCWT applied to the same retinal fundus input image as Figure 2.8. For each coefficient, its complex magnitude is computed alongside its corresponding Fourier spectrum (log-magnitude), showcasing the directional selectivity. A colour map is used to enhance the visualisation of edge structures.	14
2.12	The DTCWT-Scat module from Cotter (2020b), illustrated by Liu et al. (2022). 16	

2.13	The scattering network, from Bruna and Mallat (2013), showing iterative computation of scattering coefficients.	17
2.14	Three example images and their corresponding binary segmentation mask from DRIVE (left) and Kvasir-SEG (right).	19
3.1	Hybrid U-Net architectures with scattering transform substitutions at increasing depths. From top to bottom: the first model integrates a scattering transform with $J = 1$ (WSN-UNet-J1), the second with $J = 2$ (WSN-UNet-J2), the third with $J = 3$ (WSN-UNet-J3), and the fourth with $J = 4$ (WSN-UNet-J4). Numbers beneath network layers indicate number of feature channels. Each diagram illustrates the replacement of encoder blocks with scattering transforms, along with the corresponding modifications to skip connections. .	23
3.2	Comparison of the train loss curves for U-Net with (yellow) and without (pink) data augmentation. The loss value is plotted on the vertical axis, with the training step on the horizontal axis.	26
3.3	Comparison of the validation set dice score for U-Net with (yellow) and without (pink) data augmentation. The validation set dice score is plotted on the vertical axis, with the training step on the horizontal axis.	27
3.4	Hyperparameter Optimisation Plots for U-Net. Left: The convergence plot show the changes to negative Dice score over evaluations, reflecting the optimiser's progress. Right: The objective plot illustrates the partial dependence of each hyperparameter on the model's performance. It helps identify hyperparameters which most significantly affect Dice score.	29
3.5	Training convergence comparison between a U-Net with untuned hyperparameters (purple) and one with tuned hyperparameters (yellow). Left: Validation Dice scores throughout training. The untuned model exhibits slower generalisation and ultimately reaches a lower validation score. Right: Training loss curves. The untuned model shows less stable and slower convergence compared to the tuned version.	30
4.1	Training loss convergence for WSN-UNet-J1 and U-Net when learning on the DRIVE dataset. WSN-UNet-J1 initially converges faster for approximately the first 20 (out of 100) epochs. Loss curves are smoothed using a time-weighted exponential moving average.	34
4.2	Training loss convergence for WSN-UNet-J2 and U-Net when learning on the DRIVE dataset. WSN-UNet-J2 initially converges at a similar rate for approximately the first 15 (out of 100) epochs, but U-Net maintains a lower loss thereafter. Loss curves are smoothed using a time-weighted exponential moving average.	35
4.3	Training loss convergence for WSN-UNet-J3 and U-Net when learning on the DRIVE dataset. WSN-UNet-J3 initially converges at a similar rate for approximately the first 10 (out of 100) epochs, but U-Net maintains a lower loss thereafter. Loss curves are smoothed using a time-weighted exponential moving average.	36
4.4	Training loss convergence for WSN-UNet-J4 and U-Net when learning on the DRIVE dataset. WSN-UNet-J4 exhibits significantly slower convergence throughout training, with a consistently higher loss than U-Net across all epochs. Loss curves are smoothed using a time-weighted exponential moving average.	37

- 4.5 Percentage difference in Dice score between U-Net and WSN-UNet variants on Kvasir-SEG and DRIVE, plotted against scattering scale J . As more encoder layers are replaced (increasing J), the performance gap increases in a convex manner. Smaller performance differences on DRIVE suggest that the scattering network aids learning more effectively when training data is scarce and variability is limited. 38
- 4.6 Feature map comparisons across decoder layers for U-Net and each WSN-UNet variant, ordered left to right by decreasing performance. Rows correspond to decoder layers and columns show selected feature maps at those layers. Red boxes highlight layers where the loss of coherent structured content (top) and fine spatial detail (bottom) is most evident in lower performing models. . . . 39
- 4.7 Qualitative comparison of segmentation masks on the Kvasir-SEG dataset. Each row shows a test sample. From left to right: input image, ground truth mask, and predicted masks from selected models in descending order of performance. Higher performing models capture sharper polyp boundaries, while lower performing models exhibit artefacts and smoother borders. . . . 42
- 4.8 Qualitative comparison of segmentation masks on the DRIVE dataset. Each row shows a test sample. From left to right: input image, ground truth mask, and predicted masks from each model in descending order of performance. Red boxes highlight areas of interest where fine spatial detail is better preserved in high performing models and progressively lost in lower performing ones. . . . 42

List of Tables

3.1	Data Augmentation	25
3.2	Hyperparameter Search Space	28
4.1	Test metrics for WSN-UNet-J1 compared to U-Net on Kvasir-SEG and DRIVE.	33
4.2	Test metrics for WSN-UNet-J2 compared to U-Net on Kvasir-SEG and DRIVE.	35
4.3	Test metrics for WSN-UNet-J3 compared to U-Net on Kvasir-SEG and DRIVE.	36
4.4	Test metrics for WSN-UNet-J4 compared to U-Net on Kvasir-SEG and DRIVE.	37
4.5	Test metrics for all models on Kvasir-SEG. All metrics are reported as mean $\pm$ standard deviation. Models in red are implementations of models proposed in existing literature, all others are ours.	41
4.6	Test metrics for models on DRIVE. All metrics are reported as mean $\pm$ standard deviation. Models in red are implementations of models proposed in existing literature, all others are ours.	41
A.1	Optimal hyperparameter values for each model, selected using Bayesian optimisation. Models in red are implementations of proposed models in existing literature, all others are ours.	49
B.1	Bayesian 95% credible intervals for metrics of all models on Kvasir-SEG. Intervals are reported as (lower bound, upper bound) and rounded to 3.s.f. Models in red are implementations of models proposed in existing literature, all others are ours.	50
C.1	Comparison of Kvasir-SEG segmentation performance (Dice score) with and without concatenating the input RGB channels to the first decoder skip connection in U-Net.	51
D.1	Test metrics comparing WSN-UNet-J2 through J4 with and without intermediate skip connections on Kvasir-SEG.	52

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Georgios Exarchakis, for his invaluable guidance throughout the year, and to everyone in our weekly supervision meetings for the inspiration they provided. This project made use of Hex, the GPU Cloud in the Department of Computer Science at the University of Bath.

Chapter 1

Introduction

Image segmentation involves the labelling of different regions of an image into distinct object classes. Commonly, this is performed on a per-pixel basis known as semantic segmentation, as illustrated in Figure 1.1. Semantic segmentation is a key area of machine learning research, with numerous real-world applications. In biomedical imaging, it enables automated and precise localisation of anatomical structures, aiding disease diagnosis. In autonomous vehicles, accurate segmentation is essential for tasks such as lane detection or obstacle avoidance, where safety is critical. Image segmentation has been achieved through a variety of machine learning architectures, with successful implementations stemming from deep convolutional neural network (CNN) solutions.

Deep CNNs, consisting of stacked convolutional layers, provide an end-to-end learned solution for image segmentation. These networks learn hierarchical representations of the input image by capturing increasingly abstract spatial patterns. Convolutional layers, in combination with pooling layers, help build representations that are invariant to common variabilities such as translation or rotation. Krizhevsky, Sutskever and Hinton (2012) suggest the early layers of such networks capture these simple geometric variabilities regardless of network initialisation.

This raises the question: is it necessary to learn these generic early layers, or can they be replaced with fixed pre-defined mathematical transforms that encode these invariances? If this were the case, we could potentially achieve comparable performance with less network depth and fewer training samples - particularly beneficial in medical imaging where datasets are often limited.

The scattering network, introduced by Bruna and Mallat (2013), is a wavelet-based transform



Figure 1.1: Semantic image segmentation, adapted from Chen et al. (2019)

that provides stability to deformations and invariance to translation. It can be used as a fixed non-learned feature extractor that replaces the early layers of CNNs. Oyallon, Belilovsky and Zagoruyko (2017) demonstrated the effectiveness of such an approach in classification tasks, showing that their *hybrid* CNNs, which replace early convolutional layers with the scattering transform, outperform conventional CNNs particularly with little training data.

In the field of biomedical imaging, labelled datasets for segmentation are often small, requiring extensive data augmentation for CNNs to generalise well (Ronneberger, Fischer and Brox, 2015). This motivates the exploration of hybrid segmentation architectures. By embedding geometric invariances into the network, the hybrid network can reduce its dependence on large, augmented datasets and improve data efficiency.

In this dissertation we investigate the use of hybrid scattering CNNs for medical image segmentation. Building on Bruna and Mallat (2013)'s work, we propose several novel networks that integrate scattering networks into U-Net (Ronneberger, Fischer and Brox, 2015). We explore the extent to which the U-Net encoder's learned layers can be replaced by a scattering network, and evaluate the impact on segmentation accuracy, convergence speed, and data efficiency. Our models are primarily evaluated on the Kvasir-SEG polyp segmentation dataset (Jha et al., 2020), which contains 1000 annotated images. To investigate how performance is affected by dataset size, we additionally test on the much smaller DRIVE retinal blood vessel segmentation dataset (Staal et al., 2004), which contains only 40 annotated images.

Finally, to contextualise scattering-based hybrid CNNs within broader wavelet-CNN integrations, we investigate alternative models that replace U-Net's max-pooling and up-convolutional layers with both real-valued and complex wavelet transforms, and compare their performance against the hybrid paradigm.

Chapter 2

Literature and Technology Survey

In this chapter, we review existing literature on convolutional neural networks (CNNs) for image segmentation, with a focus on the U-Net architecture and recent enhancements. We also explore the application of wavelet transforms and scattering networks in deep learning, explaining the theoretical foundations and practical integration into CNNs.

2.1 CNNs for Image Segmentation

Deep CNNs, first introduced by LeCun et al. (1989), learn to represent input image data by extracting features such as edges, textures, and more abstract patterns. These multi-layer networks process a 2-D image by convolving kernels over the outputs of previous layers. These learnable kernels are optimised during backpropagation, enabling the network to produce feature maps that highlight critical patterns in the data. Importantly, deep CNNs are capable of learning a degree of translation invariance, allowing them to recognise objects regardless of their position in the image. This is learned implicitly by the network with the assistance of data augmentation. Activation functions transform the output at each layer, enabling the network to learn non-linear and complex patterns. Additionally, pooling layers are occasionally added to reduce spatial dimensions while retaining essential features, enhancing translation invariance and reducing overfitting. See Figure 2.1 for an illustrative figure of this process.

Following multiple convolutional layers, a final convolutional layer maps the set of feature maps to a set of n segmentation maps, where n denotes the number of labels we are segmenting. Each segmentation map assigns a value to each pixel which indicates the probability of that

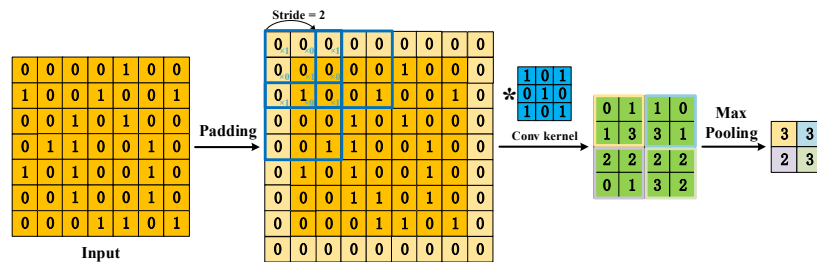


Figure 2.1: Abstracted illustration of a convolutional layer followed by a max-pooling operation from Li et al. (2021)

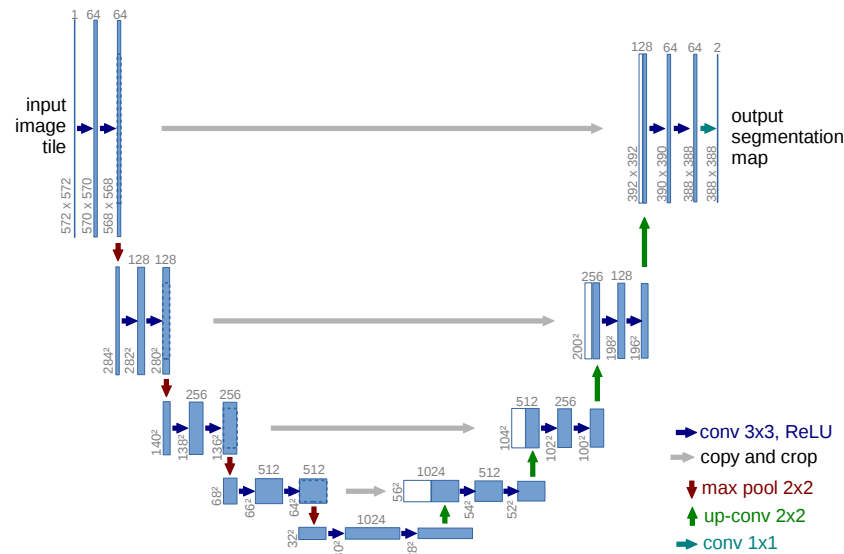


Figure 2.2: The U-Net architecture diagram, from Ronneberger, Fischer and Brox (2015). As described in the original: Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

pixel belonging to that label. In the case of binary segmentation, where we are segmenting our input image into one label category, we apply a threshold (i.e. > 0.5) to values in the segmentation map to produce a segmentation mask comprising pixel values $p \in \{0, 1\}$. During training, the CNN will learn to output segmentation map values that are higher or lower than this threshold.

2.2 The Evolution of Segmentation Architectures

A breakthrough image segmentation architecture developed by Ronneberger, Fischer and Brox (2015) is U-Net, particularly suited for datasets with limited training samples, which is common in the biomedical world. Building upon *Fully Convolutional Networks for Semantic Segmentation* by Long, Shelhamer and Darrell (2015), U-Net consists of a symmetric contracting path and expanding path of convolutional layers. The contracting path extracts contextual features using repeated 3x3 convolutions (with ReLU activation) and 2x2 max pooling to gradually reduce spatial resolution. After the contracting path, the expanding path restores spatial resolution with up-convolution operations and further 3x3 convolutions. Notably, skip connections link corresponding layers in the contracting and expanding paths to retain spatial details and localisation. The final output is a segmentation map that indicates the object classes each pixel is most likely to belong to. The U-Net architecture (see Figure 2.2) has stood the test of time and remains to be the prevalent underlying framework for many new and novel image segmentation networks today.

Over time, several significant enhancements to U-Net have been proposed to improve its performance. A highly cited architecture known as UNet++ was introduced by Zhou et al. (2018). Motivated by the strict segmentation requirements in the medical imaging industry, they proposed a series of nested, dense skip pathways between the contracting and expanding

paths, replacing the direct skip connections of U-Net. UNet++ creates multiple intermediate connections at each level that each generate increasingly more refined features. The intermediate pathways are densely connected, meaning features from all previous layers at a given depth are concatenated and used as input for the next refinement state. Through this combined reformulation, UNet++ is capable of reducing the semantic gap between encoder and decoder features, improving accuracy and generalisation.

The notion of using skip connections to shortcut information deeper through a network is also found in the ResNet (He et al., 2016) architecture. He et al. propose the use of shortcut connections over blocks (usually 2-3 layers) of the network to address the phenomenon of degradation: increasing network depth can lead to optimisation difficulties where model performance decreases, though this is not due to overfitting as training error also increases. The ResNet architecture provides a framework for the construction of very deep models, while mitigating the degradation problem this usually causes. Exploiting deeper models has been demonstrated to reliably produce leading models on many datasets, and is therefore desirable. ResNet reformulates the network to learn residual mappings (Equation 2.1)

$$F(x) = H(x) - x \quad (2.1)$$

instead of directly fitting the underlying mapping $H(x)$. This reformulation simplifies the learning as the residual function, $F(x)$, generally has smaller magnitudes and "residual" adjustments to learn, as it only needs to learn how the output deviates from the input. By performing an element-wise addition between the input to a block and the block's output as shown in Figure 2.3, it is easier to optimise the residual function than the original unreferenced mapping. By using these shortcut connections consecutively throughout the network, information can flow deeper while being less obscured by transformations.

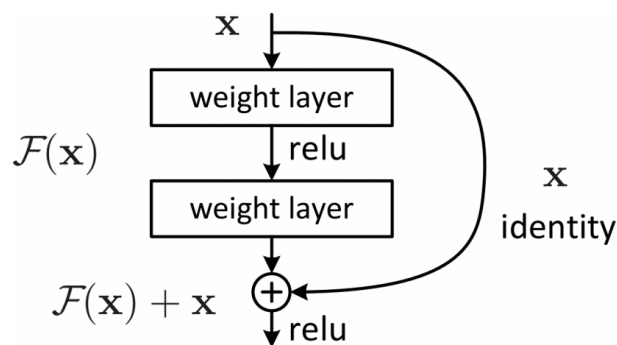


Figure 2.3: ResNet shortcut connection over block. Taken from He et al. (2016)

ResNet has been shown to generalise well to segmentation tasks, achieving 1st place in the 2015 COCO image segmentation competition with a 101-layer architecture comprising 3-layer blocks. The residual structure enabled effective training of this very deep model, which significantly outperformed existing state-of-the-art VGG (Simonyan, 2014) networks.

Residual blocks can be merged with existing image segmentation architectures. Zhang, Liu and Wang (2018) presented an architecture named ResUNet which added a residual connection over each encoder and decoder block (two convolutional layers) to construct a U-Net-like network. They found the residual units eased the training of the network, and in combination with the skip connections between the encoder and decoder layers, this facilitated information propagation without the degradation issue. They showed with the inclusion of residual blocks, it

is possible to train smaller networks with fewer parameters that perform comparably with plain architectures. During their experiments on a pixel-level road segmentation task, their ResUNet model achieved a superior breakeven point (combination metric of precision and recall) over standard a U-Net. Crucially, the ResUNet achieved this despite having a quarter as many parameters as U-Net, demonstrating the significance of the improved information propagation. However, they did not experiment with alternative datasets, and only provided results with one performance metric, making a fair comparison with alternative image segmentation models difficult.

Jha et al. (2019) extend ResUNet with their architecture ResUNet++. Through their attempt to improve polyp segmentation accuracy on the Kvasir-SEG dataset (Jha et al., 2020), they introduce three key enhancements:

1. **Squeeze and Excitation Units:** Increases sensitivity to important features.
2. **Atrous Spatial Pyramidal Pooling (ASPP):** Captures multi-scale contextual information by applying convolutions with different dilations.
3. **Attention Units:** Attention is used to guide the model to focus on the most important regions of feature maps.

Jha et al. (2019) report improved segmentation performance over ResUNet and U-Net on the Kvasir-SEG dataset, suggesting that ResUNet++ offers a preferable alternative for medical image segmentation tasks. However, their reliance on manual hyperparameter searching, potentially too coarse for a model of this depth and complexity, introduces some ambiguity about the optimality and reproducibility of their results.

2.3 Recent Novel Approaches in Segmentation

Image segmentation research continues to uncover more powerful models year-on-year that provide state-of-the-art performance on many competitive datasets. Here, we have selected several recent and novel approaches related to the Kvasir-SEG dataset (Jha et al., 2020), including top-performing models highlighted on the <https://paperswithcode.com/sota/medical-image-segmentation-on-kvasir-seg> leaderboard, as well as other notable contributions to the field.

Firstly, there have been numerous attempts to combine attention mechanisms with convolutional networks (Mohammed, 2022; Chen et al., 2021; Dong et al., 2021). Attention mechanisms assist the learning by capturing the importance of features within the context of the other features. This can be thought of as a form of feature refinement. Mohammed introduces a model called ResAttUNet which combines ideas from ResUNet and Convolutional Block Attention Module (CBAM) (Woo et al., 2018). Mohammed's approach appends a CBAM block to the end of each residual block which infers an attention map in 2-D. This addition contains two modules which refine features based on their inter-channel and inter-spatial relationships respectively: the Channel Attention Module and the Spatial Attention Module. After benchmarking ResAttUNet against standard U-Net and an 'Attention UNet' variation, ResAttUNet demonstrated conclusively it was the top-performing model across all 15 evaluation metrics on an image segmentation task. The impressive results are particularly interesting as they exhibit attention being successfully utilised outside of the language modelling domain.

The Transformer (Vaswani et al., 2017) has driven major advancements in language modelling

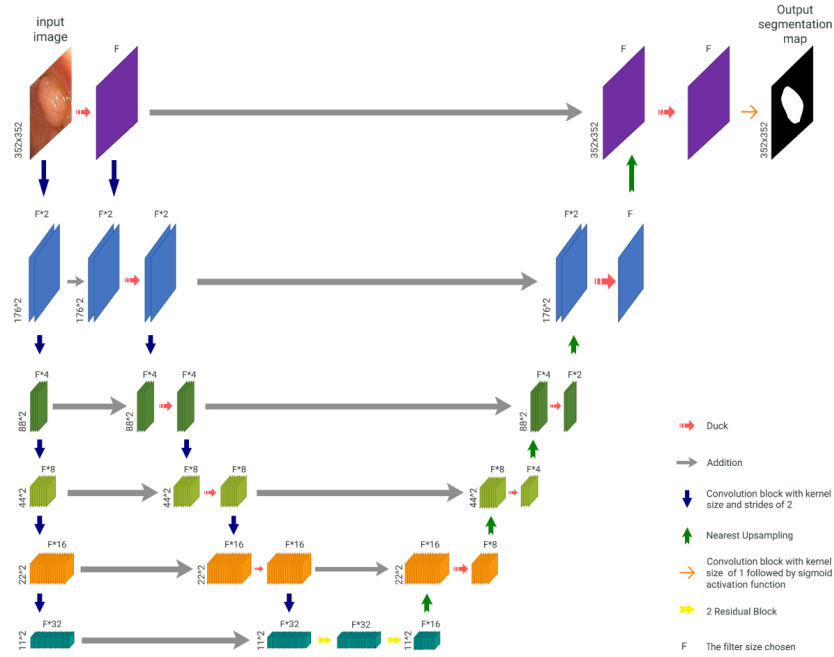


Figure 2.4: The DUCK-Net architecture diagram, from Dumitru, Peteleaza and Craciun (2023)

and generative AI. Building on its success, Sanderson and Matuszewski (2022) proposed FCBFormer, an architecture that combines traditional CNNs with the global context modelling capabilities of Transformers. Transformer-based segmentation architectures had previously shown comparable performance to convolutional approaches, however the resulting segmentation maps were typically lower resolution than the input images. Sanderson et al.'s FCBFormer solves this problem by using two parallel branches for the Transformer (pretrained on ImageNet) and the convolutional branch (Fully Convolutional Branch), each extracting features from the input image before their outputs are fused. Their results on Kvasir-SEG demonstrate a significant improvement across all metrics compared to existing Transformer and fully convolutional models.

The top performing model on Kvasir-SEG at the time of writing is DUCK-Net (Dumitru, Peteleaza and Craciun, 2023). DUCK-Net (Figure 2.4) is similar to U-Net, however it differs in two ways: double convolutions are replaced with a DUCK block (Deep Understanding Convolutional Kernel), and a secondary downscaling block which runs parallel to the contracting path. The DUCK block uses six parallel variations of convolutional blocks including residual, midscope, and widescope blocks. This allows the network to train whichever variation works best. The secondary downscaling block runs parallel to the contracting path of the U-Net, preserving low-level image details, and feeding into the main network at each layer by addition. Through experimentation of large (34 filters) and small (17 filters) versions of the network, DUCK-Net demonstrated strong generalisation and outperformed existing models (including transformer-based approaches) across all metrics on Kvasir-SEG.

2.4 Applications of Wavelets in Deep Learning

2.4.1 Convolution with the Fourier Transform

In the field of data analysis, we require mathematical tools that can find and understand the content of natural signals. Fundamentally, these tools must reduce the dimensionality of signals to remove irrelevant noise and find a better representation of the signal. A better representation must be discriminative, meaning different input signals must have unique representations. Crucially, the tools must also produce identical representations for the same input signals, even if they are not identical (Waldspurger, 2017).

One mathematical tool for this is the Fourier Transform which decomposes a signal into a linear combination of its frequency components. The discrete Fourier transform (DFT) of a 1D signal $x[n]$ of length N is defined as:

$$\mathcal{F}[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-2\pi i k n / N}, \quad k = 0, 1, \dots, N-1 \quad (2.2)$$

The DFT can be extended for higher dimensional signals such as 2D images.

In image processing, convolution is an operation used to apply filters to images that highlight particular features such as edges or textures. These filters are called kernels and can be designed to extract certain features, or learned in the context of CNNs.

Convolution theorem states that convolution in the spatial domain is equivalent to pointwise multiplication in the frequency domain. So, for two signals $x[n]$ and $h[n]$, their convolution $y[n] = (x * h)[n]$ is computed by:

$$\mathcal{F}\{x * h\}[k] = \mathcal{F}\{x\}[k] \cdot \mathcal{F}\{h\}[k] \quad (2.3)$$

Therefore, the convolution can be computed by first applying the Fast Fourier Transform (FFT) (Cooley and Tukey, 1965) to both the image and kernel, then performing pointwise multiplication in the frequency domain, and finally applying the inverse FFT to return to the spatial domain. This process reduces the time complexity of convolution from $\mathcal{O}(N^2)$ in the spatial domain, to $\mathcal{O}(N \log N)$ in the frequency domain. Figure 2.5 shows how an image and kernel are transformed to the frequency domain, multiplied, and then transformed back with the inverse FFT to produce an equivalent convolved output.

2.4.2 Wavelet Transforms

Continuous Wavelet Transform

The Fourier transform decomposes a signal into an infinite sum of sine and cosine waves. This provides insights into the different global frequency components that occur within the signal. However, the transform provides no information about when each frequency occurred. For non-stationary signals (where the frequency components change in time), it would be useful to know not only what frequencies are present, but also when they happened. Continuous wavelet transforms (CWT) address this by providing a way of analysing a signal in both the time and frequency domains.

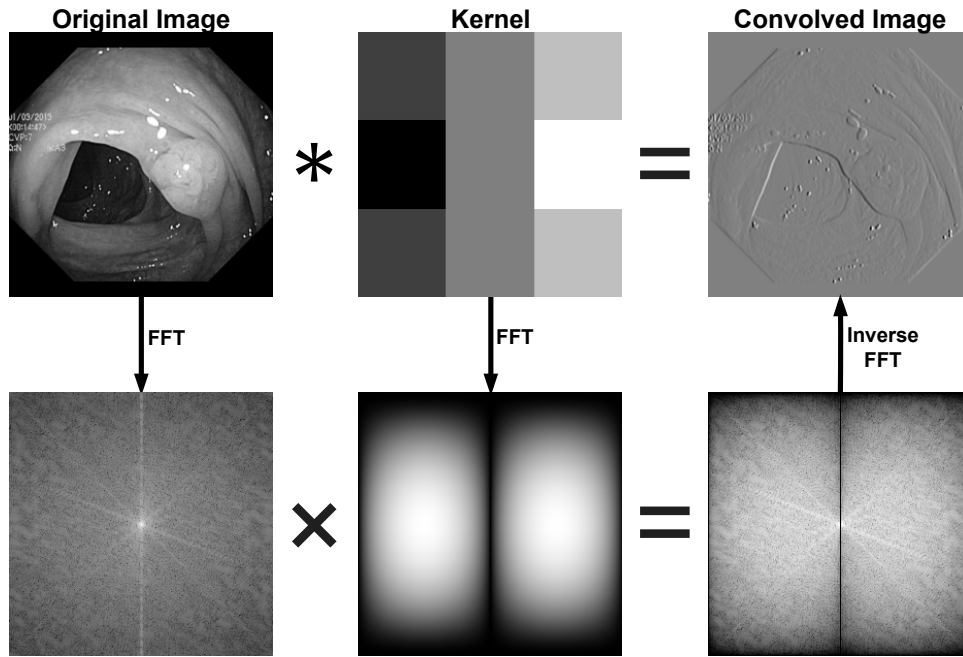


Figure 2.5: Convolution with the Fourier Transform. The input and Sobel kernel are transformed to the frequency domain, multiplied pointwise, and transformed back with the inverse FFT.

As discussed in Mallat (1999)'s *A Wavelet Tour of Signal Processing*, a wavelet $\psi \in \mathbf{L}^2(\mathbb{R})$ is a function defined with a zero mean and finite energy:

$$\int_{-\infty}^{+\infty} \psi(t) dt = 0, \quad \int_{-\infty}^{+\infty} |\psi(t)|^2 dt < \infty.$$

We can obtain a set of daughter wavelets by scaling and translating ψ :

$$\mathcal{D} = \left\{ \psi_{u,s}(t) = \frac{1}{\sqrt{s}} \left(\frac{t-u}{s} \right) \right\}_{u \in \mathbb{R}, s \in \mathbb{R}^+} \quad (2.4)$$

Then, the wavelet transform of a signal $f \in \mathbf{L}^2(\mathbb{R})$ with time u , and scale s is defined as the inner product of f and $\psi_{u,s}$:

$$Wf(u, s) = \langle f, \psi_{u,s} \rangle = \int_{-\infty}^{+\infty} f(t) \frac{1}{\sqrt{s}} \psi^*\left(\frac{t-u}{s}\right) dt \quad (2.5)$$

The transform effectively outputs a coefficient, which represents the similarity between signal f , and the daughter wavelet $\psi_{u,s}$. By varying s and u , we can produce a two-dimensional scalogram, which maps the wavelet coefficients over a time-scale plot, revealing how frequency content evolves, as shown in Figure 2.6.

While the Fourier transform provides perfect frequency resolution, it has no temporal localisation. The wavelet transform trades some frequency resolution for some time resolution. This trade-off is governed by the Heisenberg uncertainty principle (Equation 2.6): as we increase our certainty in time, this reduces our certainty in frequency and vice versa. This is visible in a scalogram as the spread of the ridge in either time or frequency axes. Mallat illustrates this in Figure 2.7.

$$\sigma_t^2 \sigma_\omega^2 \geq \frac{1}{4\pi} \quad (2.6)$$

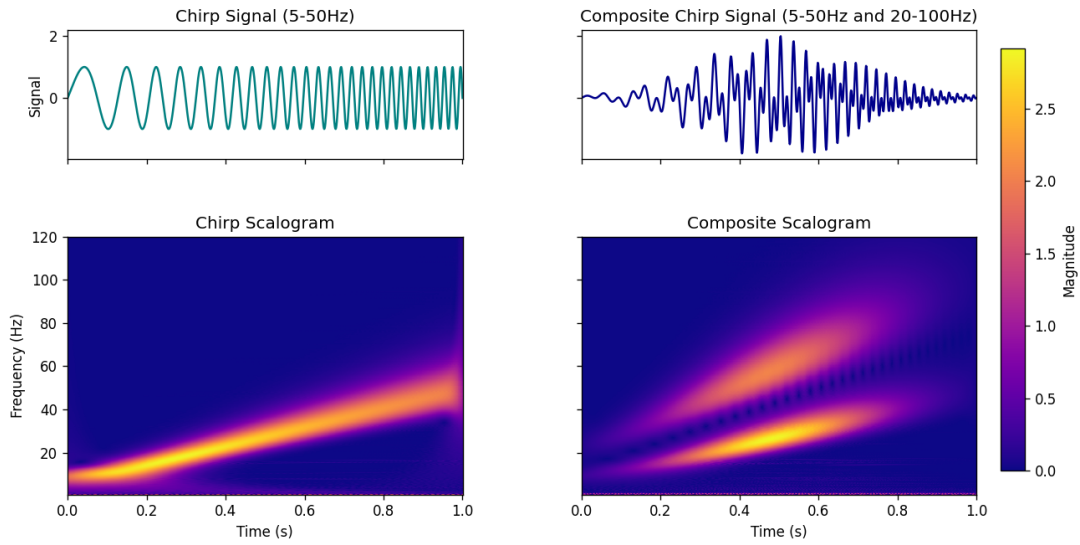


Figure 2.6: Comparison of time-domain signals and corresponding continuous wavelet transforms (using the complex Morlet wavelet) for a single chirp (left) and a composite chirp signal (right). The single chirp increases linearly from 5-50 Hz, while the composite signal includes an additional chirp from 20-100 Hz. Note how the CWT distinctly separates the two superimposed signals in the time-frequency domain.

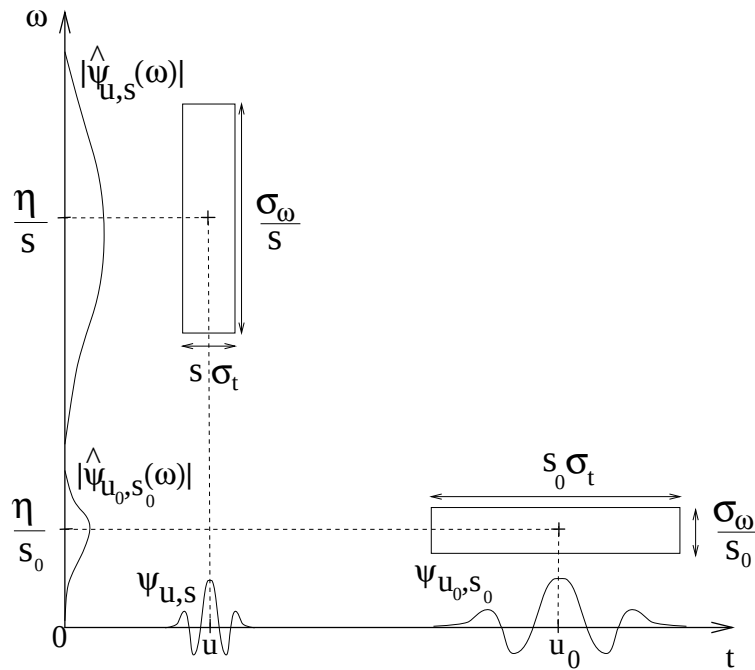


Figure 2.7: Heisenberg uncertainty boxes from Mallat (1999). Each box illustrates the time-frequency localisation of a wavelet at a given scale. Smaller scaled wavelets result in finer time resolution, but coarser the frequency resolution. The opposite is true for larger scales.

Discrete Wavelet Transform

The CWT provides a highly redundant representation of a signal, as it computes coefficients at all possible scales and translations. This results in unnecessary repetition and correlation between coefficients. In 2-D, this inefficiency becomes more pronounced: as noted by Cotter (2020b), the CWT yields a four-dimensional coefficient space (two spatial dimensions, one for scale, and one for orientation).

To address this Mallat (1989) presents the discrete wavelet transform (DWT), a modification to the CWT which samples at discrete scales and translations dyadically. This produces a non-redundant representation which is efficient to compute with a filter bank tree, and can be reconstructed with an inverse transform. A 1-D daughter wavelet used in the DWT is defined:

$$\psi_{j,l}(t) = \frac{1}{\sqrt{a}} \psi\left(\frac{t-b}{a}\right) = 2^{-j/2} \psi(2^{-j}t - l) \quad (2.7)$$

where $a = 2^j$ and $b = 2^j l$ control the scale and translation respectively. j is the scale factor, it controls the granularity of our scale and ensures our representation is not redundant.

Following Cotter (2020b)'s description, the 2-D DWT uses a scaling function and three wavelet functions composed as the product of 1-D wavelets in both horizontal (u_1) and vertical (u_2) directions:

$$\text{Scaling: } \phi(\mathbf{u}) = \phi_{j,l}(u_1)\phi_{j,m}(u_2) \quad (2.8)$$

$$\text{Horizontal detail: } \psi_{j,l,m}^H(\mathbf{u}) = \phi_{j,l}(u_1)\psi_{j,m}(u_2) \quad (2.9)$$

$$\text{Vertical detail: } \psi_{j,l,m}^V(\mathbf{u}) = \psi_{j,l}(u_1)\phi_{j,m}(u_2) \quad (2.10)$$

$$\text{Diagonal detail: } \psi_{j,l,m}^D(\mathbf{u}) = \psi_{j,l}(u_1)\psi_{j,m}(u_2) \quad (2.11)$$

The indices $l, m \in \mathbb{Z}$ control horizontal and vertical translations, respectively. Applying each of these to the image with an inner product produces the corresponding wavelet coefficients.

In practice, it is efficient to compute this using a filter bank which is functionally equivalent to the theory above. Mallat (1989) presents a pyramid architecture to compute the DWT of a 2-D signal, illustrated in Figure 2.9. At each level, the input is convolved with a low (L) and high-pass (H) filter that approximate the daughter wavelets defined above. The low-pass filter corresponds to the scaling function ϕ , producing the approximation coefficients (LL), while the high-pass filters correspond to the wavelet functions, yielding the detail coefficients: LH (horizontal detail), HL (vertical detail), and HH (diagonal detail). After filtering, the outputs are downsampled by a factor of two, this mirrors the dyadic sampling and ensures a non-redundant representation as illustrated in Figure 2.8.

Dual-Tree Complex Wavelet Transform

The DWT suffers from two limitations: poor translation invariance and directional selectivity. This motivates the Dual-Tree Complex Wavelet Transform (DTCWT) by Kingsbury (1998), which addresses these issues by using complex wavelets constructed via a pair of parallel filter banks similar in architecture to the DWT. In the DTCWT, these separate filter banks independently compute the real and imaginary parts of complex coefficients. It is designed such that the wavelet filters form approximate Hilbert pairs, enabling their outputs to simulate the behaviour of an analytic wavelet whose Fourier spectrum is concentrated in the positive

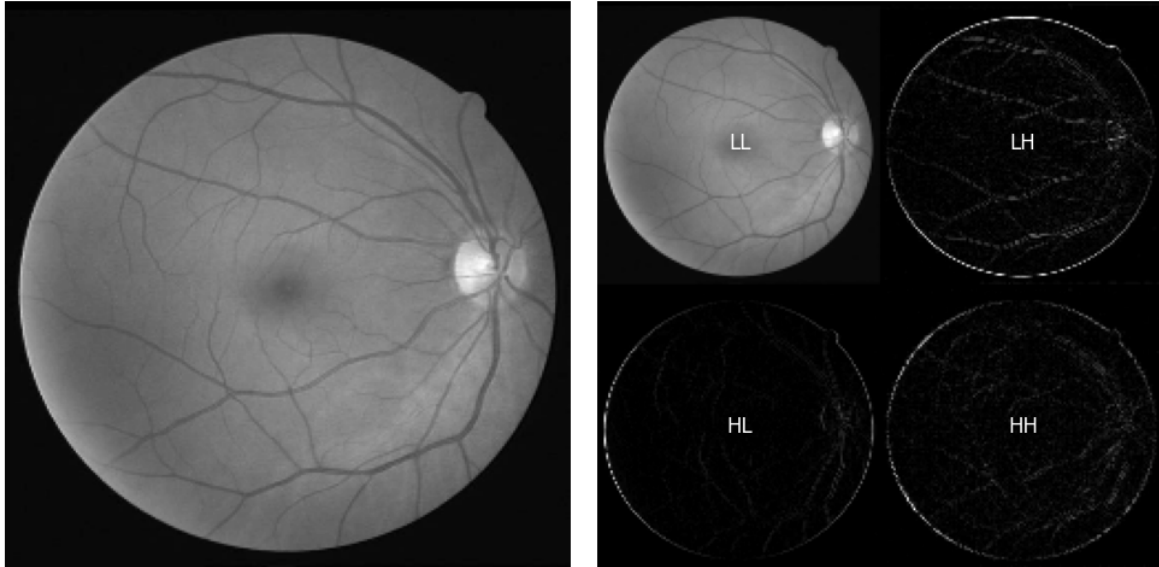


Figure 2.8: The discrete wavelet transform ($J=1$) applied to a retinal image. The left image shows the input, and the right is an arrangement of the four labelled coefficients. Both the left and right images are the same resolution, demonstrating the non-redundancy of the transform.

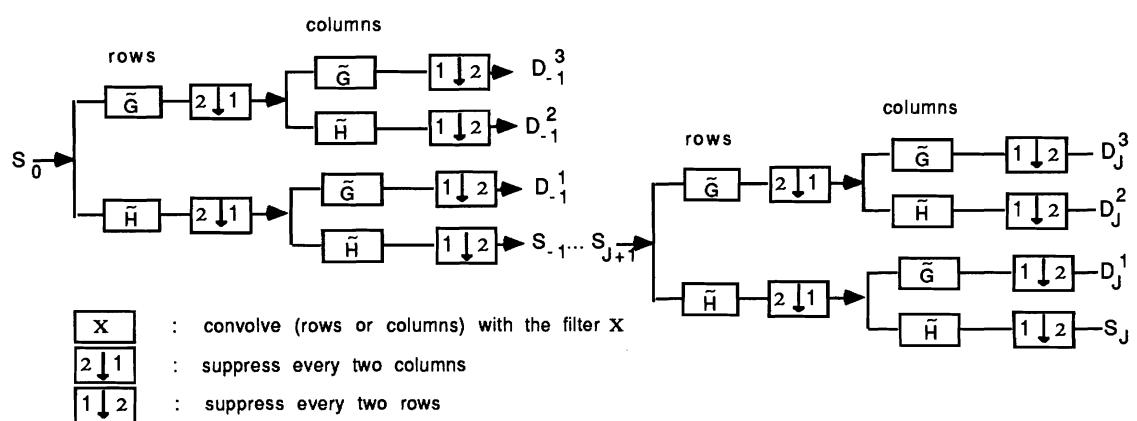


Figure 2.9: Pyramid Architecture for the 2-D DWT, from Mallat (1989)

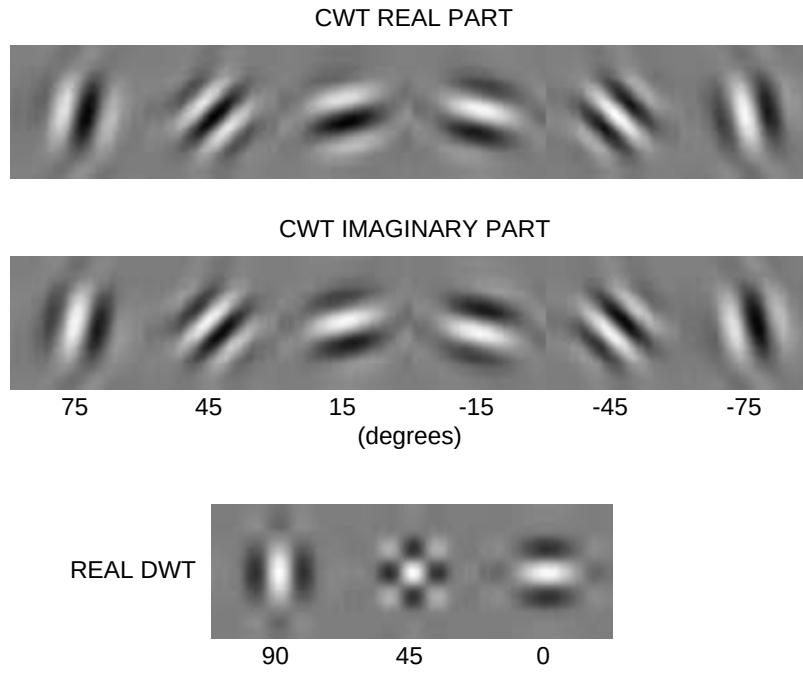


Figure 2.10: 2-D impulse responses of DTCWT, and impulse responses from DWT. Figure taken from Kingsbury (1998).

frequencies. Kingsbury (1998) show this results in approximate translation invariance, as the energy of complex coefficients is more stable under small input shifts than the DWT.

We apply separable filtering along rows and columns to compute coefficients for 2-D signals, resulting in six orientated subbands per level at angles of $\pm 15^\circ$, $\pm 45^\circ$, and $\pm 75^\circ$. This provides a more comprehensive angular resolution than the three directions of the DWT. The impulse responses of these wavelets, shown in Figure 2.10, resemble Gabor wavelets which have the best theoretical trade-off between frequency and spatial localisation.

The output of the DTCWT is a set of six complex coefficient maps per level, with each coefficient emphasising features aligned with the orientation of its corresponding wavelet filter, as illustrated in Figure 2.11. The combination of improved translation invariance, orientation sensitivity, and embedded phase information makes the DTCWT a powerful feature extractor, prompting its integration with CNNs.

2.4.3 Wavelet Enhanced U-Net for Segmentation

Among the numerous applications of both the DWT and DTCWT, is their use as feature extractors within deep CNNs. When applied with a scale of $J = 1$, both transforms produce coefficients that are half the spatial resolution of the input - mirroring the behaviour of max-pooling and up-convolution operations of U-Net. This makes wavelet transforms a compatible, enhanced replacement for downsampling and upsampling layers. Moreover, the sub-band decompositions inherently capture directionally selective features, and in the case of the DTCWT, the transform introduces approximate translation invariance. These characteristics motivate their integration into segmentation networks like U-Net.

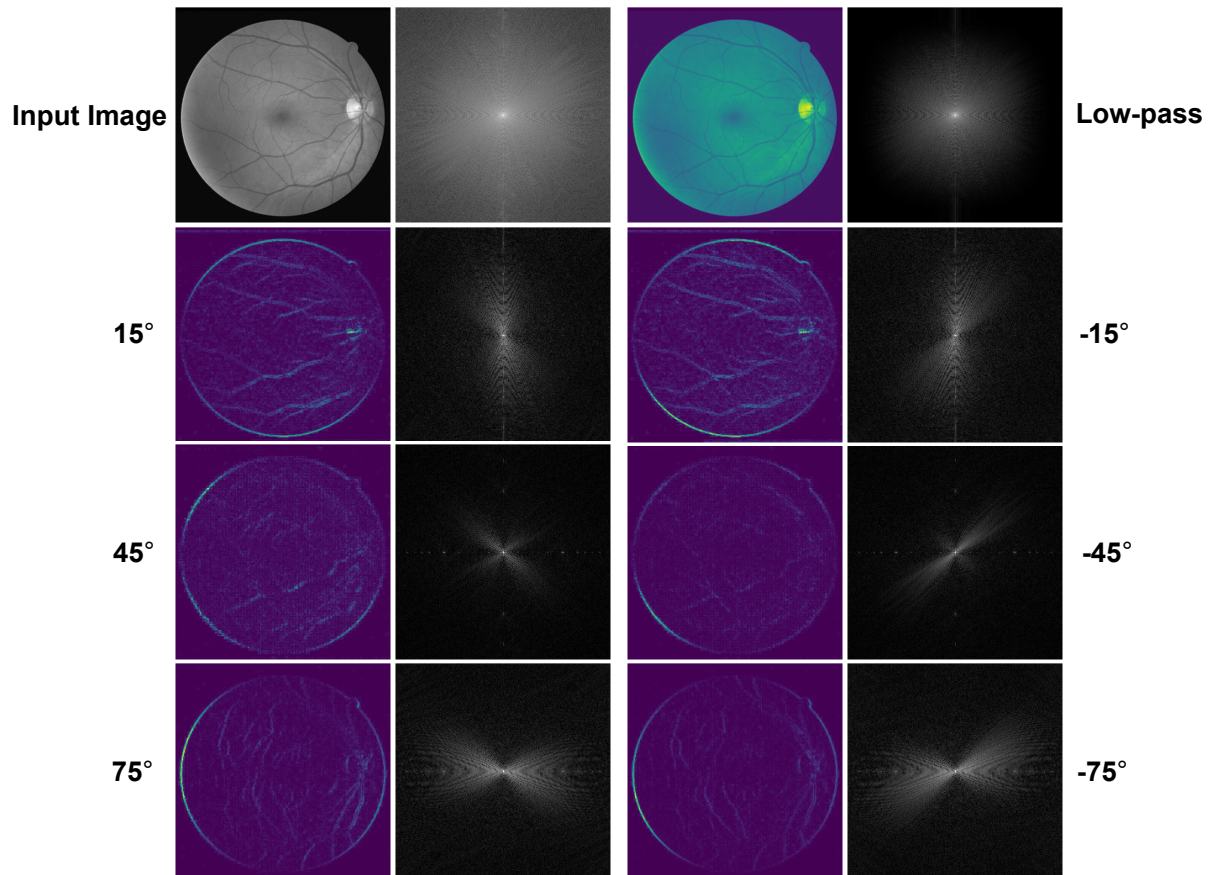


Figure 2.11: The DTCWT applied to the same retinal fundus input image as Figure 2.8. For each coefficient, its complex magnitude is computed alongside its corresponding Fourier spectrum (log-magnitude), showcasing the directional selectivity. A colour map is used to enhance the visualisation of edge structures.

Alijamaat, NikravanShalmani and Bayat (2021) propose a wavelet enhanced U-Net by replacing max-pooling layers with the DWT and upsampling layers in the decoder with the inverse DWT (IWT). In their study on brain lesion segmentation, they compare wavelet-pooling modules to traditional max-pooling and average-pooling. They demonstrate significant performance improvements across all evaluation metrics when using wavelet-based pooling. Their results compared performance over different sized lesions, and wavelet-pooling was superior over all sized lesions.

This work is further extended by Agnes, Solomon and Karthick (2024), who integrated DWT pooling and upsampling into U-Net++. Evaluating both standard U-Net and U-Net++ alongside their wavelet-enhanced counterparts on a lung nodule segmentation dataset, they again show that wavelet-based pooling consistently outperforms conventional pooling. They attribute this improvement to the transform's efficient extraction of multi-scale features and preservation of significant spatial details.

Liu et al. (2022) leverage the advanced directional selectivity and approximate translation invariance of the DTCWT by replacing the max-pooling of the U-Net encoder with the DTCWT. Their model, LSW-Net, incorporates the DTCWT-Scat module (Figure 2.12) proposed in Cotter (2020b). In experiments on both a brain tumour segmentation dataset and a retinal blood vessel segmentation dataset, their model consistently outperforms all classical segmentation architectures evaluated, including the standard U-Net. Although their study does not directly compare against the DWT-based wavelet pooling approach proposed by Alijamaat, NikravanShalmani and Bayat (2021), they conclude that the DTCWT is particularly effective at resolving weak boundaries and capturing fine structural details, such as narrow blood vessels.

Peng, Sonka and Chen (2024) introduce Spectral U-Net which retains complex-valued DTCWT coefficients, rather than computing their modulus as done in Cotter (2020b). In scattering, the complex modulus introduces non-linearity and translation invariance, while discarding phase information. Spectral U-Net preserves both magnitude and phase throughout the network, with downsampling and upsampling performed via the DTCWT and its inverse, followed by tensor rearrangements. This is a lossless process, unlike traditional max-pooling. Despite a 16-fold increase in encoder feature counts (4 from the low-pass and 12 from the sub-bands), Spectral U-Net maintains a comparable parameter count to a standard U-Net, because the decoder reduces feature counts by the same factor. Spectral U-Net achieves consistent improvements over U-Net across three datasets.

2.5 Wavelet Scattering Networks

2.5.1 Theoretical Foundation of Wavelet Scattering Networks

The scattering network introduced by Mallat (2012) uses repeated applications of the wavelet transform modulus to produce a set of scattering coefficients that are invariant to translation and stable to deformations. Extending this work, Bruna and Mallat (2013) presented a complete scattering convolution network in their paper *Invariant Scattering Convolution Networks*.

Shortcomings in the performance of image classification models stem from the inability of euclidean distance to measure similarity and variability within image classes. Such variability is introduced by both rigid transformations (e.g. translation, rotation, scaling) and non-rigid transformations (e.g. subtle deformations like variations in handwriting styles for digit recognition). While achieving invariance to deformations is undesirable, since deformations

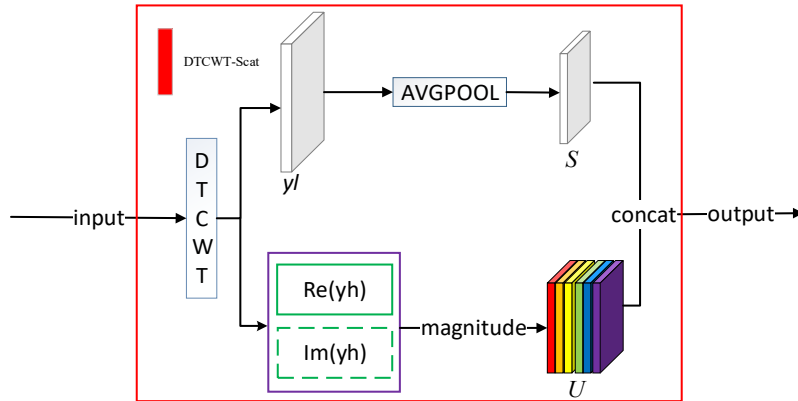


Figure 2.12: The DTCWT-Scat module from Cotter (2020b), illustrated by Liu et al. (2022).

often encode valuable class information, models must handle these transformations effectively to ensure robust classification (Bruna and Mallat, 2013).

Invariance to translation can be computed with Fourier transform modulus, but this approach lacks stability to deformations. Wavelet transforms, however, are inherently stable to deformations due to their localisation properties, but they are covariant to translation rather than invariant. Building invariant representations from wavelet coefficients is possible using deep CNNs, where learned filters and non-linear operators extract invariants that are stable to deformation. However, deep CNN architectures require a lot of training data and need to optimise a large number of parameters which is computationally demanding and slow. Bruna and Mallat (2013) proposed an alternative approach using the scattering transform, a mathematical method that cascades wavelet transforms across a filter bank over multiple rotations and scales, as illustrated in Figure 2.13. This process produces a robust set of features without the need for explicit learning, addressing the limitations of deep CNNs while retaining stability to deformations and capturing meaningful invariants.

As described by Waldspurger (2017), the scattering transform is constructed through an iterative cascade of wavelet transform modulus operations. Given a function f , a family of wavelets $(\psi_j)_{j \in \mathbb{Z}}$, and a low-pass filter ϕ_J with $\int \phi_J(t) dt = 1$, the scattering transform is defined over the set of paths:

$$P = \{(j_1, \dots, j_k), k \in \mathbb{N}, j_1 \leq J, \dots, j_k \leq J\}, \quad (2.12)$$

where J is a parameter defining the maximum scale of the transform used to restrict low-frequency content.

For any path $p = (j_1, \dots, j_k) \in P$ with length $|p| > 0$, the scattering coefficient is computed as:

$$S[p]f = |\dots ||f * \psi_{j_1}| * \psi_{j_2}| \cdots * \psi_{j_k}| * \phi_J \quad (2.13)$$

For the empty path ($|p| = 0$), the coefficient is simply:

$$S[p]f = f * \phi_J, \quad (2.14)$$

known as the zeroth order scattering coefficient. This captures global components of f .

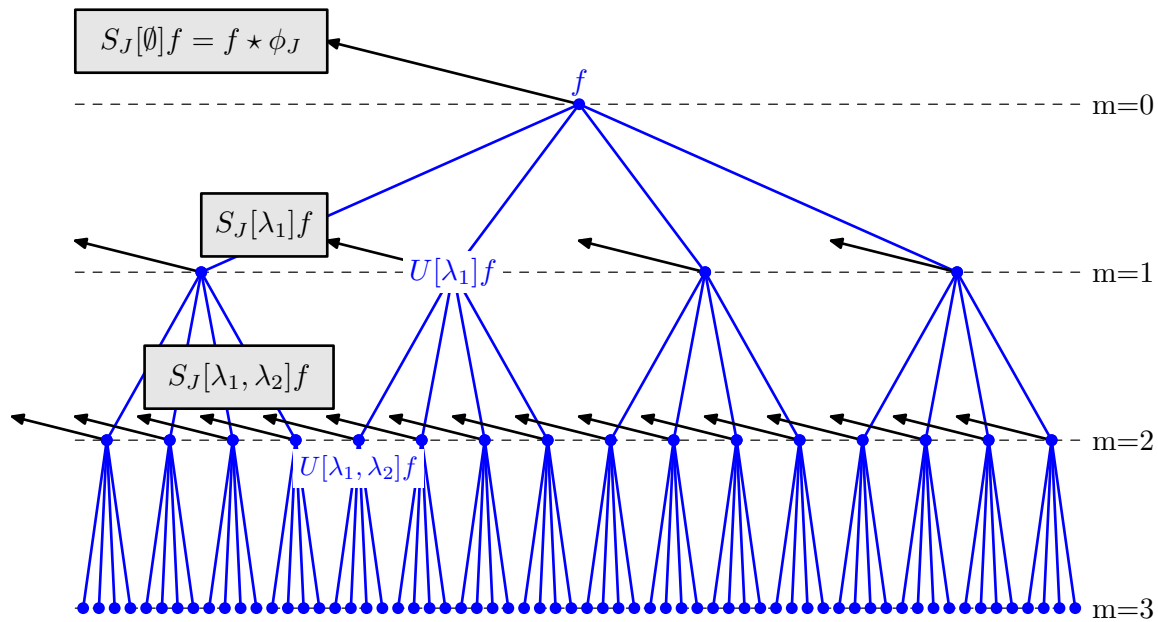


Figure 2.13: The scattering network, from Bruna and Mallat (2013), showing iterative computation of scattering coefficients.

The complete scattering transform S of f is then defined as the collection of all coefficients:

$$Sf = (S[p]f)_{p \in P} \quad (2.15)$$

Wavelets at scales j_n and all orientations θ_n (e.g. $\{0, 60, 120\}$) are applied in each layer, so long as the scale has increased from the previous operation on that path. Each layer of the scattering network provides higher-order scattering coefficients of the signal by applying the wavelet transforms, modulus operation, and scaling function. The first layer captures local features at different scales and orientations, while each subsequent layer captures increasingly higher-order features by applying the same operations to the output of the previous layer.

The scattering network can be modified to handle various types of transformations such as rotation. Sifre and Mallat (2013) introduced a framework for rotation, scaling, and deformation invariance in scattering networks, focusing on texture discrimination. While they achieved full rotation invariance by averaging scattering coefficients over multiple orientations, Oyallon and Mallat (2015) took a different approach with their deep roto-translation scattering network for object classification. Rather than imposing full rotation invariance, which would be too strong a prior for object recognition tasks, they developed a separable wavelet transform along both spatial and angular variables that linearises rotation variability. This resulted in an "augmented SIFT" representation that proved effective for object classification tasks, achieving comparable results to unsupervised deep learning methods on datasets such as Caltech and CIFAR.

2.5.2 Deep Hybrid Networks: Combining Wavelet Scattering Networks with CNNs

It is understood that the early layers of convolutional neural networks learn kernels that can extract features based on frequency and orientation information (Krizhevsky, Sutskever and

Hinton, 2012). It is difficult to specify the exact purpose of these learned filters, however it is clear that end-to-end trained CNNs are capable of learning invariances to geometrical transformations such as translations and rotations. The invariances that are learned are beneficial, as they eliminate variabilities in image classes that are not discriminative. Transfer learning has shown that with networks trained on large datasets, the learned filters are capturing generic properties useful for classification tasks (Huh, Agrawal and Efros, 2016). Oyallon, Belilovsky and Zagoruyko (2017) pose the question of whether these early layers need to be learned, or whether they can be replaced by pre-defined non-learned filters. They investigate the impact of replacing the early layers of CNN architectures with scattering networks on a variety of architectures and datasets.

Oyallon et al. note that on smaller, simpler datasets where variabilities are known (such as MNIST), the scattering transform can yield excellent results. However, on more complex datasets such as CIFAR where the intra-class variabilities are more subtle and non-geometrical, the scattering transform falls short of learned representations. They argue each layer of a CNN implicitly approximates increasingly complex Lie groups of symmetry, and that the roto-translation group $\mathbb{R}^2 \rtimes SO_2$ is commonly learned. Hence, rather than learning this symmetry from scratch, pre-defined scattering transforms can explicitly model it, allowing subsequent layers to focus on learning more abstract variabilities. This motivates the hybrid network, with the hope that the CNN layers can reduce the more complex non-geometrical variabilities that scattering layers are unable to filter. Additionally, replacing layers of CNNs reduces depth and the spatial dimensions of layers, potentially leading to faster and more interpretable end-to-end learning pipelines.

For subsets (100, 500, 1000) of the CIFAR-10 dataset, Oyallon et al. demonstrated that a Wide ResNet with scattering could outperform plain Wide ResNet and VGG-16 by a significant margin. For a limited number of samples, this is expected due to the geometric priors in the scattering networks. Promisingly, this shows the improved convergence rates with pre-defined scattering layers.

They also directly compared a ResNet-18 to a ResNet-10 with scattering. Both models achieved analogous performance (88.6 and 88.8 Top 5 respectively) on Imagenet ILSVRC-2012 with a similar number of parameters. These results demonstrate the scattering network was capable of replacing the early layers of the CNN without degrading the model's discriminative ability. Although these results were achieved on classification tasks, an extension of the hybrid architecture for image segmentation shows great promise.

2.6 Segmentation Datasets

The performance of any learned model is ultimately evaluated using benchmark datasets. Selecting an appropriate dataset is important, and considerations about its size, variability, applications, and ethics contribute to the final choice. In this project, datasets were chosen for their relevance to medical imaging tasks and their use in existing literature, allowing direct comparison with published models.

2.6.1 Kvasir-SEG

The Kvasir-SEG dataset (Jha et al., 2020) is a collection of 1000 images extracted from colonoscopy procedures. Colorectal cancer (commonly known as bowel cancer) affects the

large intestine and is a prevalent form of cancer in both men and women. The presence of polyps (precancerous growths) in the large intestine indicates an increased risk of colorectal cancer, and these polyps are usually surgically removed to mitigate this. The detection of such polyps is therefore a crucial step in the prevention of bowel cancer. Colonoscopy is a procedure that allows a medical professional to assess the inside of the colon and rectum for any polyps using an inserted camera. However, some studies show there is an approximate miss rate of 14-30% (Van Rijn et al., 2006) where polyps are unnoticed due to their size and obscurities. Deep learning offers a solution to improve polyp detection, and Kvasir-SEG was developed to support this. The dataset comprises 1000 colonoscopy images containing at least one polyp and a corresponding set of 1000 binary segmentation mask images (see Figure 2.14). The segmentation masks are images of equivalent resolution to their counterpart where white coloured pixels indicate the presence of a polyp, and all other pixels are black. Kvasir-SEG provides a relevant, competitive, and challenging testbed for image segmentation.

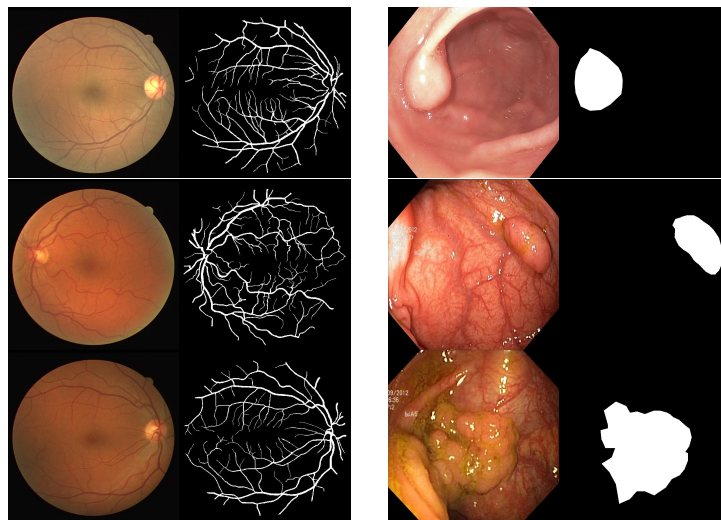


Figure 2.14: Three example images and their corresponding binary segmentation mask from DRIVE (left) and Kvasir-SEG (right).

2.6.2 DRIVE: Retinal Blood Vessel Segmentation

The DRIVE (Digital Retinal Images for Vessel Extraction) (Staal et al., 2004) is another binary segmentation dataset, focused on the segmentation of blood vessels in retinal fundus images, illustrated in Figure 2.14. Retinal imaging is crucial in the diagnosis of many diseases including diabetic retinopathy, where the appearance, width, and branching of the blood vessels provide important indicators.

Unlike the Kvasir-SEG dataset which involves segmenting relatively large contiguous polyp regions, the DRIVE dataset presents a highly contrasting segmentation task. Retinal blood vessels are thin with complex, branching structures. This requires more sensitivity to fine spatial and directional detail. Moreover, DRIVE contains only 40 annotated images, with 20 allocated to both training and testing. This is considerably smaller than Kvasir-SEG, making it an ideal benchmark for assessing the robustness of models where data is limited.

Chapter 3

Methodology

In this chapter, we explain the experimental design used to investigate the integration of wavelet transforms into U-Net for medical image segmentation. We detail the research questions, proposed models, dataset preparation, hyperparameter tuning, training procedure, and evaluation metrics used in the project.

3.1 Research Questions

This project investigates the use of scattering transforms as a replacement for the learned encoder path in U-Net, focusing on whether their built-in translation invariance and stability to deformation can provide meaningful low-level representations without the need for training. Specifically, we explore whether scattering-based encoders can match or outperform fully learned encoders, and to what extent they support accurate segmentation. We replace increasing portions of the U-Net encoder with scattering transforms at different scales and assess their impact on segmentation performance, training efficiency, and convergence. By analysing loss curves and evaluation metrics, we aim to understand how much of the encoder can be replaced without sacrificing task-specific details.

We also investigate alternative uses of wavelet transforms by replacing U-Net’s downsampling and up-sampling operations with fixed wavelet transforms (DWT and DTCWT), assessing their effect on model performance. The following questions guide the project:

1. **To what extent can scattering networks replace learned encoder layers in U-Net, and how does the depth of this replacement affect segmentation performance and training efficiency?**

This question examines whether scattering transforms can serve as drop-in replacements for early convolutional layers and whether deeper integration leads to better or worse outcomes in terms of both performance and training dynamics.

2. **Do hybrid scattering-U-Net architectures offer improved data efficiency, particularly with limited training data?**

We evaluate model performance on both the large Kvasir-SEG dataset and the smaller DRIVE dataset to determine whether scattering-based hybrids generalise better with limited training data

3. How do DWT and DTCWT compare as replacements for U-Net’s downsampling and up-sampling operations, and what is the most effective way to integrate wavelet transforms into the encoder-decoder structure.

This question assesses whether fixed wavelet operations can enhance the efficiency or accuracy of the network, and which transform (if any) is best suited to this role.

3.2 Experiment Design

This work investigates the integration of scattering and wavelet transforms into the U-Net architecture to evaluate their influence on segmentation performance. Building on existing literature, we propose several modified U-Net models, which are trained and evaluated on two datasets to analyse how these transformations affect performance.

3.2.1 Proposed Model Architectures

Our baseline model is U-Net (Ronneberger, Fischer and Brox, 2015), as detailed in section 2.2, page 4. To enable useful comparison across experiments, we implemented this first. We also hypothesised that U-Net performance could be improved by concatenating the original RGB input channels with the first skip connection to the final decoder block, potentially enhancing spatial context and reconstruction accuracy. This minor modification was applied across all experiments, and its impact on performance is shown in Appendix C.

We then proposed four hybrid U-Net architectures, WSN-UNet-J1 through WSN-UNet-J4, where WSN denotes *Wavelet Scattering Network* and J indicates the maximum scale used in the scattering transform. These models, described in subsequent sections, each incorporate the scattering transform at a different fixed maximum scale. In each case, a portion of the U-Net encoder is replaced by a scattering transform with a specified maximum scale J , while the remainder of the network retains the standard U-Net structure. Each model replaces a progressively deeper portion of the U-Net encoder, allowing us to examine how the extent of replacement affects performance. The four models use scattering transforms at $J = 1$ through $J = 4$. The spatial dimensions of scattering coefficients shrink by a factor 2^J as a result of the transform, as illustrated in Equation 3.1. This aligns precisely with the spatial dimensions of feature maps in the U-Net encoder, which are halved at each stage via max-pooling, allowing the seamless integration of the scattering output.

$$H' = \frac{H}{2^J}, \quad W' = \frac{W}{2^J} \quad (3.1)$$

In all four hybrid architectures, we restore skip connections to the decoder by passing scattering coefficients directly into the decoder pathway. When multiple skip connections are required, the coefficients are separated by their scattering order. First-order coefficients are inserted closer to the final decoder layers, while second-order coefficients are inserted at earlier decoder blocks. First-order coefficients capture localised frequency content such as edges and textures, whereas second-order coefficients represent interactions between them, analogous to the abstract representations formed in cascading CNN layers. Each set of coefficients is passed through its own up-convolutional operation to match the spatial dimensions required at its respective decoder block. These skip connections act as small parallel decoders at varying depths for scattering coefficients, enhancing their representation before merging.

WSN-UNet-J1

WSN-UNet-J1 replaces the initial double convolution and first max-pooling layer of the U-Net encoder with a scattering transform of $J = 1$ and $L = 16$ orientations. This halves the input spatial dimensions, matching the first U-Net max-pooling output.

The scattering coefficients are forwarded to the second encoder block, while the original RGB input is concatenated with the scattering coefficients and used as the skip connection to the final decoder block. We use a single up-convolutional layer to ensure the scattering output matches the size of the input tensor before concatenation. This restores spatial information in the decoder and improves segmentation accuracy. The architecture is illustrated in the top network of Figure 3.1.

WSN-UNet-J2

WSN-UNet-J2 uses a scattering transform with $J = 2$ and $L = 8$, replacing the first two encoder blocks, and reducing the input dimensions by a factor of 4.

The scattering coefficients are forwarded into the third encoder block. Original skip connections are restored by separating the scattering output by orders. Zeroth- and first-order coefficients are upsampled via a double up-convolution layer and concatenated with the RGB input channels, creating the skip connection to the final decoder block. Second-order coefficients extract and represent more abstract feature maps, these are passed through an up-convolution layer and skipped to the penultimate decoder block. This structure is shown in the second diagram of Figure 3.1.

WSN-UNet-J3

WSN-UNet-J3 increases the scattering scale to $J = 3$ and $L = 8$, which replaces the first three encoder blocks, and reduces the input dimensions by a factor of 8.

The scattering coefficients are forwarded to the fourth encoder block. Again, zeroth- and first-order coefficients are upsampled (via a triple up-convolution) and concatenated with the input channels for the final decoder block. First order-coefficients are upsampled with a double up-convolution and skipped to the penultimate decoder block. Second-order coefficients are passed through a single up-convolution layer and are skipped to the second decoder block. This network is illustrated in the third diagram of Figure 3.1.

WSN-UNet-J4

WSN-UNet-J4 uses a scattering transform at $J = 4$ and $L = 8$ which replaces the entire U-Net encoder. This reduces the input dimensions by a factor of 16, and feeds into the double convolutional layer bridge before the decoder.

Skip connections are restored in the same way as in WSN-UNet-J3, however we use an extra up-convolutional layer in each skip connection as the scattering output has half the spatial dimensions. We also restore the final skip connection by skipping the second-order coefficients to the first decoder block. This network is illustrated in the bottom diagram of Figure 3.1.

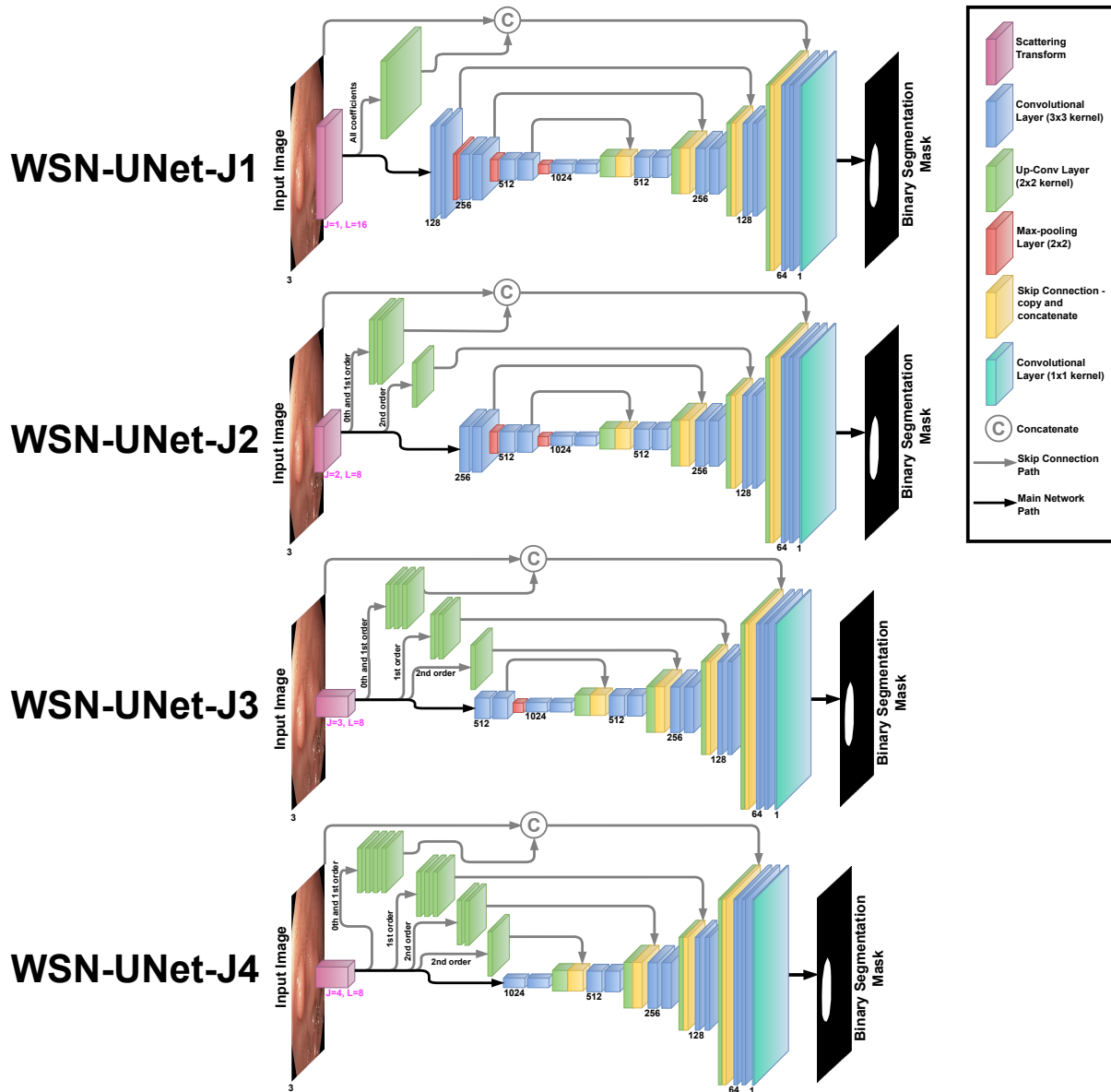


Figure 3.1: Hybrid U-Net architectures with scattering transform substitutions at increasing depths. From top to bottom: the first model integrates a scattering transform with $J = 1$ (WSN-UNet-J1), the second with $J = 2$ (WSN-UNet-J2), the third with $J = 3$ (WSN-UNet-J3), and the fourth with $J = 4$ (WSN-UNet-J4). Numbers beneath network layers indicate number of feature channels. Each diagram illustrates the replacement of encoder blocks with scattering transforms, along with the corresponding modifications to skip connections.

Validating Skip Connections

To justify the separation and distribution of scattering coefficients by their order to different stages of the decoder, we also train variations of the WSN-UNet-J(2-4) models which simply skip the entire scattering output to the final decoder block, without restoring skip connection to the earlier decoder blocks. The results of this comparison are shown in Appendix D.

Dual-Tree Complex Wavelet Transform Enhanced U-Net

To evaluate the impact of integrating inverse wavelet transforms into the U-Net decoder, we combine the encoder of LSW-Net (Liu et al., 2022) with two alternative decoders:

- The inverse DWT-based decoder from Alijamaat, NikravanShalmani and Bayat (2021)
- A custom inverse DTCWT-based decoder detailed below

We refer to these models as **LSW-iDWT** and **LSW-iDTCWT**, respectively. Our custom DTCWT-based decoder follows a similar design to the inverse DWT decoder of Alijamaat, NikravanShalmani and Bayat (2021), beginning with a convolutional expansion of feature channels, followed by splitting the tensor into lowpass and highpass components. The lowpass features are upsampled using transposed convolution, while the highpass coefficients are reshaped into the complex subband format required for inverse dual-tree complex wavelet transform (DTCWT). The reconstructed output is then concatenated with the skip connection and refined through standard double convolution layers. The code for the PyTorch implementation of this module is shown in Appendix F.

LSW-Net uses a standard U-Net decoder, meaning the benefits of a wavelet enhanced decoder remains unexplored for their encoder. By substituting in both types of wavelet-based decoder, we can isolate and assess the contribution of the inverse DWT and inverse DTCWT as up-sampling operators for the reconstruction of segmentation maps.

3.2.2 Dataset Preparation

As previously discussed in section 2.6 (page 18), we are primarily using the Kvasir-SEG dataset (Jha et al., 2020) to evaluate the performance of the various models we have proposed. We also evaluate models on the smaller DRIVE dataset (Staal et al., 2004). For simplicity, the following discussion is focused on the preparation of Kvasir-SEG. For a detailed comparison of the differences in data preparation and training procedure for the DRIVE dataset, see Appendix E.

Kvasir-SEG consists of two directories: one containing raw RGB colonoscopy images and the other containing corresponding segmentation masks (see Figure 2.14). Each directory contains 1000 JPEG-encoded images, with each image and its corresponding mask sharing the same filename.

To prepare the dataset for our supervised learning experiments, we first divided it into three subsets: training, validation, and testing. This preprocessing step reformatted the dataset into three directories (`test/`, `val/`, `train/`), where each contained its own `imgs/` and `masks/` subdirectories. We adopt the conventional 80:10:10 split consistent with Jha et al. (2020) ensuring sufficient training data while providing reliable evaluation. This standardised split serves as a control across all experiments, enabling fair comparisons between model architectures.

During training, batches of images are extracted from the training set to train the model. As each batch is loaded, a sequence of operations is applied to the images and the corresponding masks. First, we apply random transformations to the images and masks, known as data augmentation. This is a crucial step that helps prevent overfitting, enabling the model to generalise better to unseen data. Ronneberger, Fischer and Brox (2015) describe data augmentation as an essential tool for enabling models to learn invariances to shifts and rotations

as well as robustness to deformations. The random elastic deformations, as well as other transformations we apply to the training data, are detailed in Table 3.1. All data augmentation was implemented using the `torchvision.transforms` module from the PyTorch library (Paszke, 2019).

Table 3.1: Data Augmentation

Transform	Parameters
Random horizontal flip	With probability 0.5
Random vertical flip	With probability 0.5
Random rotation	Uniform random rotation between -45° and 45°
Colour jittering (not applied to segmentation masks)	Uniform random adjustment by up to $\pm 20\%$ to brightness, contrast, and saturation. Hue is randomly rotated within interval $[-0.1, 0.1]$

Ronneberger, Fischer and Brox (2015) highlight the limited size of biomedical datasets as a major reason for using data augmentation. While our dataset of 800 training samples did not face this issue, we still investigated the effect of data augmentation on model performance. We trained two standard U-Nets for 50 epochs, one with data augmentation, and the other without. The results indicate that without data augmentation, the model overfits (as seen in the training loss curves Figure 3.2) and fails to generalise as well to unseen data (as shown by the dice score curves Figure 3.3).

After applying data augmentation, we normalise the pixel values to lie within the range $[0, 1]$. This normalisation ensures consistent scaling of the pixel values and aids the model in learning more effectively. As highlighted by Ioffe and Szegedy (2015), normalisation techniques can enhance the convergence of deep learning models by mitigating internal covariate shift. While this preprocessing does not involve batch normalisation as described in (Ioffe and Szegedy, 2015), it supports the general principle of scaling input values for improved convergence and stability.

To ensure consistent tensor sizes within each batch during training, we resize all images and masks to 128×128 pixels using bicubic interpolation. This consistency is necessary because the training set image resolutions range from 332×487 to 1920×1072 pixels, and varying image sizes would lead to incompatible tensor shapes during gradient computation, causing errors in backpropagation. However, during testing and general use, the model can handle images of any size, as the architecture is fully convolutional and designed to process arbitrary input dimensions.

3.2.3 Bayesian Hyperparameter Optimisation

Accompanying the trainable parameters of a deep neural network are hyperparameters - pre-defined constants that govern the network's behaviour during training. Unlike the network's weights, these hyperparameters are not learned via backpropagation. Instead, they are chosen before training and remain unchanged throughout. Typical examples include the learning rate, weight decay, batch size, and network depth. Crucially, these hyperparameters can have a significant impact on model performance. For instance, the learning rate controls the step size

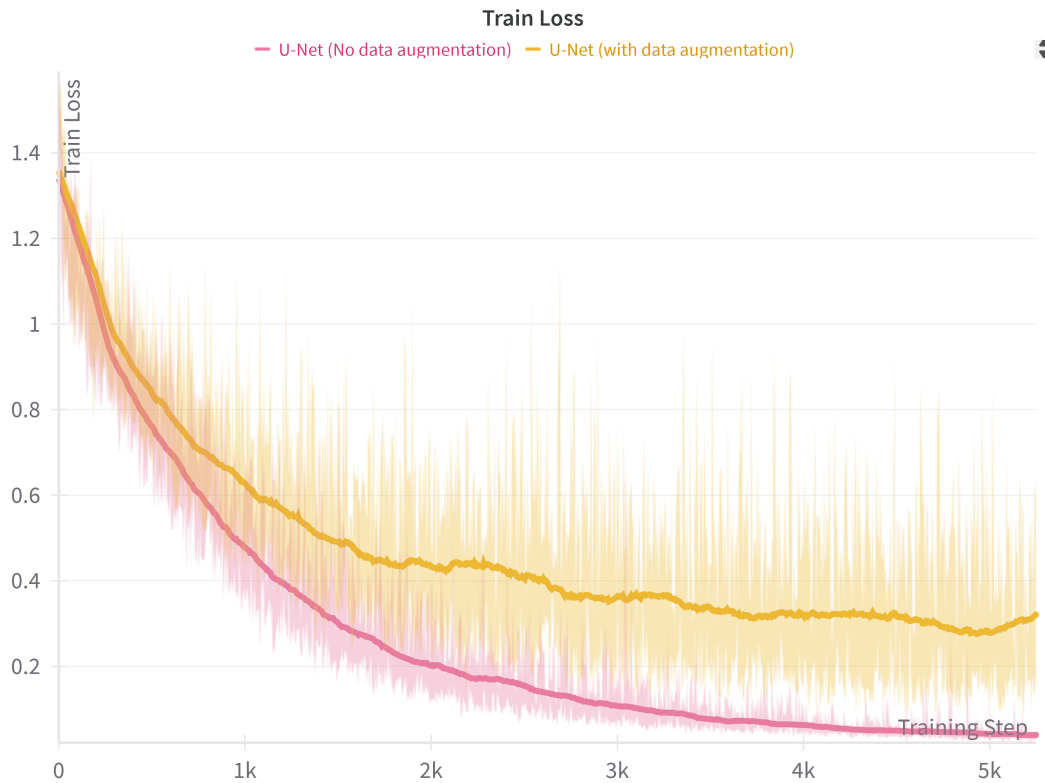


Figure 3.2: Comparison of the train loss curves for U-Net with (yellow) and without (pink) data augmentation. The loss value is plotted on the vertical axis, with the training step on the horizontal axis.

used by the optimiser during parameter updates. If this learning rate is too high, the optimiser may step over local minima, preventing the model from converging. Conversely, if the learning rate is too low, the optimiser may converge too slowly or fail to converge altogether. Therefore, to exploit the full potential of a neural network, we must attempt to optimise the available hyperparameters prior to training.

Several widely used strategies exist for tuning hyperparameters, including manual search, grid search, and random search. As discussed in Yang and Shami (2020)'s review, manual tuning relies on human intuition and trial-and-error, making it impractical for large hyperparameter spaces or costly evaluations. Grid search, a brute-force method that exhaustively evaluates predefined hyperparameter combinations, suffers from inefficiency in high-dimensional spaces due to the curse of dimensionality. In contrast, random search (Bergstra and Bengio, 2012) improves efficiency by randomly sampling the space, often finding good configurations faster than grid search when only a few hyperparameters significantly impact performance.

The Bayesian Approach

Grid search and random search both suffer from inefficient exploration of the hyperparameter space, and this is emphasised when tuning computationally expensive objective functions such as deep neural networks. These methods treat each evaluation as an independent sample and do not exploit information gained from previous evaluations.

Bayesian hyperparameter optimisation offers a sample-efficient approach which, as described

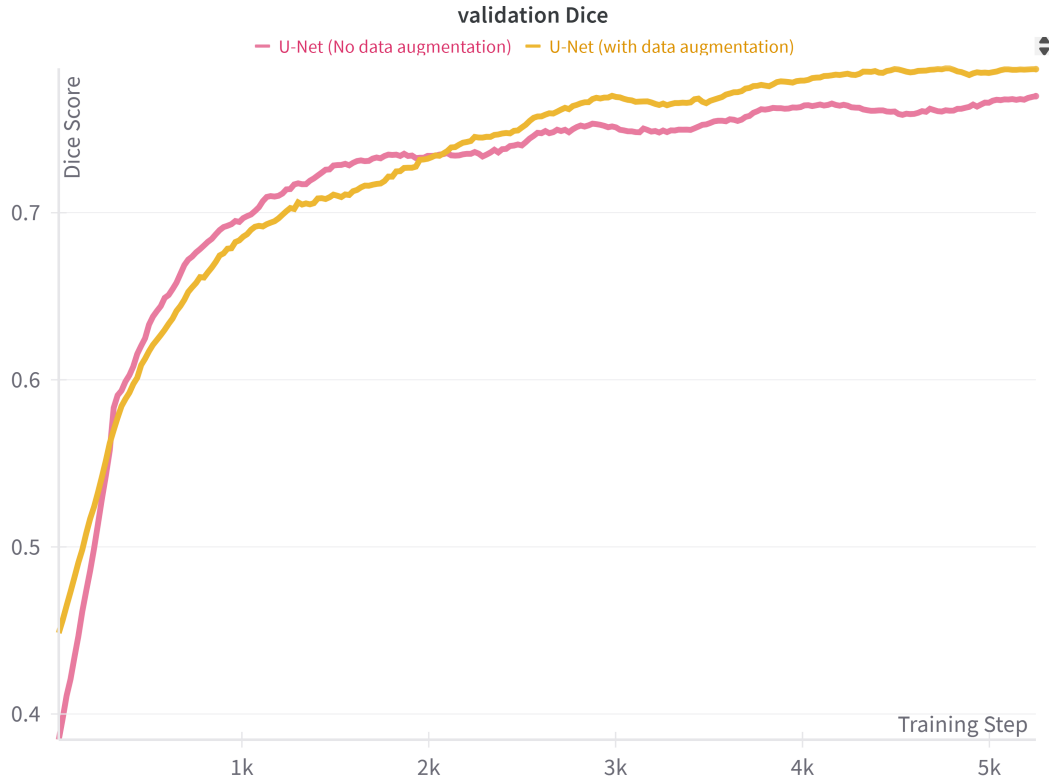


Figure 3.3: Comparison of the validation set dice score for U-Net with (yellow) and without (pink) data augmentation. The validation set dice score is plotted on the vertical axis, with the training step on the horizontal axis.

in Snoek, Larochelle and Adams (2012), models the objective function as a black-box function drawn from a prior such as a Gaussian Process. As new data is observed, Bayesian optimisation can update its belief about the objective function, and select the next hyperparameter sample based on experience, while balancing exploration with exploitation. Snoek, Larochelle and Adams (2012) demonstrated that a Bayesian optimiser can find better hyperparameters significantly faster than alternative approaches, and consequently improve state-of-the-art deep CNNs by over 3% on the CIFAR-10 dataset.

Bayesian hyperparameter optimisation models the objective function (e.g. the validation dice score as a function of hyperparameters) as a Gaussian Process (GP) prior. From Bayes' Theorem:

$$P(f | D) = \frac{P(D | f) \cdot P(f)}{P(D)} \quad (3.2)$$

the posterior $P(f | D)$ is updated iteratively with observed data $D = \{(x_i, y_i)\}_{i=1}^n$, where x_i are hyperparameters and $y_i = f(x_i)$. For a GP prior $f(x) \sim \text{GP}(0, k(x, x'))$ with kernel k , the posterior at a new point in hyperparameter space x_* becomes:

$$f(x_*) | D \sim \mathcal{N}(k_*^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y}, k(x_*, x_*) - k_*^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} k_*), \quad (3.3)$$

where $\sigma_n^2 \approx 10^{-4}$ represents the noise variance in our experiments (see standard deviations in Table 4.5). Acquisition functions, $a(x)$, such as Expected Improvement, Probability of Improvement, and Upper Confidence Bound are used in conjunction with a proxy optimisation to

determine our next hyperparameters $x_{\text{next}} = \operatorname{argmax}_x a(x)$. For instance, the proxy optimisation can be a gradient-based method, or trials of random sampling.

Implementation Details

First, we identified four universal hyperparameters for U-Net-type architectures: learning rate, weight decay, gradient clipping, and batch size. Due to memory constraints, the batch size was fixed at 8, which provides sufficient noise for effective stochastic gradient descent. With batch size fixed, the search space was reduced to three continuous hyperparameters. The search intervals for each were chosen based on established best practices and informed by an initial round of trial-and-error, as summarised in Table 3.2.

Table 3.2: Hyperparameter Search Space

Hyperparameter	Search Range
Learning rate	(1e-5, 1e-2)
Weight decay	(1e-8, 1e-2)
Gradient clipping	(0.5, 2.0)

Before training each proposed model (subsection 3.2.1), Bayesian hyperparameter optimisation was conducted using the validation set. The results of the optimisation were stored in the experiment record and used to configure the model.

Bayesian optimisation was carried out using the `gp_minimize` function from the `scikit-optimize` library (Head et al., 2018), which uses a Gaussian process to model the objective function. The optimiser was tasked with minimising the negative Dice score on the validation set. A maximum of 100 evaluations was set, balancing exploration of the hyperparameter space with computational cost of training and evaluating each configuration. A summary of the optimisation process is provided in Algorithm 1.

Algorithm 1 Bayesian Hyperparameter Optimisation Using Gaussian Processes

```

1: Input: Training and validation image/mask directories
2: Search Space:
3:   Learning rate  $\sim \log \mathcal{U}(10^{-5}, 10^{-2})$ 
4:   Weight decay  $\sim \log \mathcal{U}(10^{-8}, 10^{-2})$ 
5:   Gradient clipping  $\sim \mathcal{U}(0.5, 2.0)$ 
6: Prepare training and validation datasets with data augmentation
7: Set batch size = 8 and initialise data loaders
8: function OBJECTIVE( $\theta = (\eta, \lambda, \gamma)$ )
9:   Instantiate U-Net model on available device
10:  Train model for 50 epochs with:
11:    Learning rate  $\eta$ 
12:    Weight decay  $\lambda$ 
13:    Gradient clipping threshold  $\gamma$ 
14:  Evaluate model on validation set to get Dice score  $S$ 
15:  return  $-S$  ▷ Score is negated to minimise
16: end function
17: Run Bayesian optimisation: gp_minimise(Objective, space, 100 calls)
18: Save optimisation results to file

```

Following this optimisation process, the best hyperparameter values for learning rate, weight decay, and gradient clipping were extracted from the optimisation results, which were saved in a .pkl file. To better understand the optimisation trajectory and search dynamics, we generated three visualisations (see Figure 3.4): a convergence plot showing the negative Dice score across evaluations, an objective plot illustrating the partial dependence of each hyperparameter on model performance, and an evaluations plot revealing the distribution of sampled configurations across the search space.

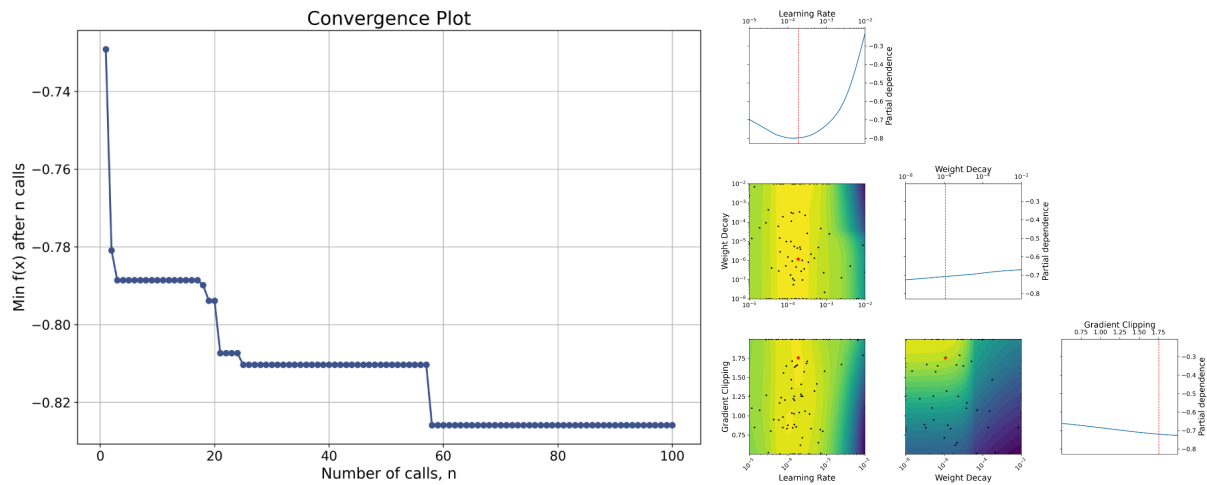


Figure 3.4: Hyperparameter Optimisation Plots for U-Net. **Left:** The convergence plot show the changes to negative Dice score over evaluations, reflecting the optimiser’s progress. **Right:** The objective plot illustrates the partial dependence of each hyperparameter on the model’s performance. It helps identify hyperparameters which most significantly affect Dice score.

The plots of the tuning results revealed that the learning rate was the most influential

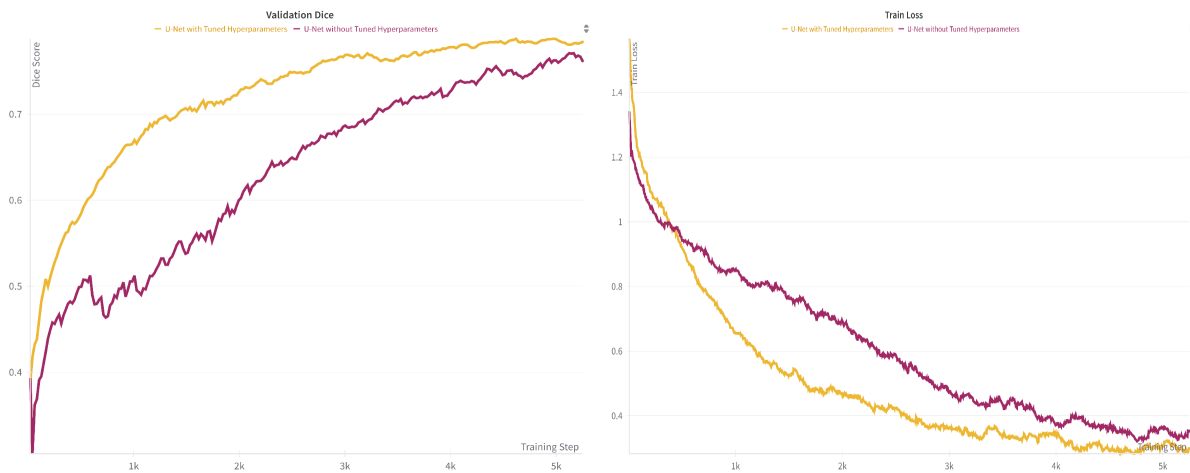


Figure 3.5: Training convergence comparison between a U-Net with untuned hyperparameters (purple) and one with tuned hyperparameters (yellow). **Left:** Validation Dice scores throughout training. The untuned model exhibits slower generalisation and ultimately reaches a lower validation score. **Right:** Training loss curves. The untuned model shows less stable and slower convergence compared to the tuned version.

hyperparameter on model performance. In contrast, the choice of weight decay and gradient clipping, when paired with an optimal learning rate, had relatively minor effects on performance. While both weight decay and gradient clipping needed to be optimised, it was the learning rate that exhibited the most pronounced partial dependence (as shown in the right plot in Figure 3.4).

To demonstrate the practical impact of hyperparameter tuning on training stability, convergence, and model performance, we compared two U-Net models: one trained using untuned (but reasonable) hyperparameters, and another trained using optimised values. The difference in training behaviour is shown in Figure 3.5, highlighting the instability and inconsistent train loss and validation curves of the untuned version.

A complete summary of the optimal hyperparameter configurations for each model is presented in Table A.1 in the appendix.

3.2.4 Training Procedure

To ensure reproducibility, robustness and interpretability of the results, our training procedure follows the best practices and recommendations by Renard et al. (2020). Given the inherent variability introduced by factors such as stochastic optimisation, random weight initialisation, hyperparameter choice, and variability within the dataset itself, it is important to adopt a rigorous experiment design. We therefore perform multiple independent training runs with the same hyperparameters for each model to capture this variability, and report the mean and standard deviations of each metric. In line with Renard et al. (2020)’s guidance to avoid biased hyperparameter selection, we use a Bayesian hyperparameter search rather than manual tuning. Finally, following the recommendation to report at least three metrics due to inter-metric correlation and task-specific relevance, we evaluate four complementary metrics: dice score, intersection over union, precision, and recall.

To robustly evaluate model performance, we perform 100 independent training runs, each lasting 50 epochs with randomly initialised model parameters. For each run, the model is trained using the AdamW optimiser with the loss function defined as the sum of binary cross-entropy and dice loss (1-dice score). Training data is randomly augmented, as described in Table 3.1. After each run, the model is evaluated on the test set, and the four performance metrics are recorded. A 95% Bayesian credible interval is computed for each metric, representing the range within which the true metric value lies with 95% posterior probability. Training is stopped early if these credible intervals narrow below a threshold for all metrics. Algorithm 2 provides a detailed breakdown of this procedure.

Algorithm 2 Training Procedure

Require: Hyperparameters H : {learning rate, weight decay, gradient clipping}, Epochs E , batch size B , maximum runs R

- 1: Initialise model object and move to device
- 2: Define image augmentations and load dataset
- 3: Initialise counters for metrics: successes and failures
- 4: **for** each run $r = 1 \dots R$ **do**
- 5: Reset model parameters
- 6: Initialise optimiser (AdamW) with H
- 7: **for** each epoch $e = 1 \dots E$ **do**
- 8: **for** each batch b in training set **do**
- 9: Apply random augmentations to b
- 10: Forward pass and compute loss:
- 11: Loss = Binary cross-entropy loss + Dice loss
- 12: Backpropagate and update weights with gradient clipping
- 13: **end for**
- 14: **end for**
- 15: Evaluate model on test set, obtain metrics
- 16: **for** each metric **do**
- 17: Update Beta posterior with observed metric scores
- 18: Compute Bayesian credible interval $[lb, ub]$
- 19: **end for**
- 20: Log test results with credible intervals to file
- 21: **if** all credible intervals satisfy $ub - lb < 0.01$ **then**
- 22: **break** early
- 23: **end if**
- 24: **end for**

Implementation and Computational Resources

All models were implemented using the *PyTorch* deep learning framework (Paszke, 2019). The wavelet scattering network was implemented using *Kymatio* (Andreux et al., 2020), while wavelet transforms (DWT and DTCWT) were implemented using *Pytorch Wavelets* Cotter (2020a), which supports gradient backpropagation. Links to our development code and datasets are provided in Appendix G.

All experiments were trained on either NVIDIA RTX 3090 (24GB) or RTX 2080 (8GB) GPUs, depending on availability and computational requirements.

3.2.5 Evaluation Strategy and Performance Metrics

At the end of each training iteration, we evaluate and record the performance of the trained model. As explained in subsection 3.2.2, we use a pre-defined test set of 100 unseen images from the Kvasir-SEG dataset to do this.

Each test image is processed by the trained model to produce a predicted segmentation mask, which is then compared to the true mask. We compute four metrics to evaluate the quality of the prediction. These metrics are averaged over all 100 test images to give an overall evaluation of the model, testing generalisation to unseen data. Descriptions and computation of each metric are explained below:

Dice Score : Dice score measures the overlap between predicted and true segmentation masks, and is useful for evaluating performance on imbalanced datasets where there is a disproportion between positive class and negative class pixels.

$$\text{Dice Score} = \frac{2TP}{2TP + FP + FN} \quad (3.4)$$

Intersection over Union : Intersection over Union (IoU) also measures the overlap between the predicted and true segmentation masks. It penalises mismatches more heavily than Dice score.

$$\text{IoU} = \frac{TP}{TP + FP + FN} \quad (3.5)$$

Precision : Precision measures the proportion of predicted positive pixels that are true positives. A high precision indicates a low false positive rate.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3.6)$$

Recall : Recall measures the proportion of true positive pixels that were correctly identified by the model. A high recall rate indicates a low false negative rate.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3.7)$$

After computing each metric over all 100 training runs, we then calculate the mean and standard deviation for each metric. This enables us to quantify the uncertainty of our model evaluation and present results with greater clarity and reliability than by training and testing each model once. This method does however require heavy computation, and would be unreasonable for much larger datasets or complex models.

Chapter 4

Results and Discussion

In this chapter, we present and discuss the quantitative and qualitative results of the proposed models, benchmarked against published model implementations. We evaluate both segmentation performance and data efficiency on the Kvasir-SEG dataset and on the smaller DRIVE dataset.

4.1 Hybrid Scattering Models

This section presents and analyses the performance of the four proposed hybrid wavelet scattering models in comparison to the baseline U-Net. We evaluate each model on both the Kvasir-SEG and DRIVE datasets, reporting the mean and standard deviation of four segmentation metrics. In addition to quantitative results, we examine training loss convergence curves to assess data efficiency in the low-data regime that DRIVE provides. Qualitative analysis is also conducted by comparing predicted segmentation masks to ground truth masks, highlighting areas where models encounter difficulties.

4.1.1 WSN-UNet-J1

WSN-UNet-J1 replaces the first two convolution layers of U-Net with a wavelet scattering transform of scale $J = 1$, as described in section 3.2.1 (page 22). Table 4.1 presents a comparison of WSN-UNet-J1 against our baseline U-Net across four segmentation metrics on both the Kvasir-SEG and DRIVE datasets.

Table 4.1: Test metrics for WSN-UNet-J1 compared to U-Net on Kvasir-SEG and DRIVE.

Dataset	Model	Dice	IoU	Precision	Recall
Kvasir-SEG	U-Net	0.843 ± 0.0142	0.765 ± 0.0162	0.854 ± 0.0288	0.886 ± 0.0238
	WSN-UNet-J1	0.838 ± 0.0151	0.761 ± 0.0168	0.861 ± 0.0328	0.870 ± 0.0320
DRIVE	U-Net	0.786 ± 0.0135	0.649 ± 0.0168	0.845 ± 0.0196	0.745 ± 0.0329
	WSN-UNet-J1	0.779 ± 0.0188	0.642 ± 0.0223	0.823 ± 0.0505	0.758 ± 0.0417

On both datasets, WSN-UNet-J1 achieves performance that is comparable to U-Net, despite having two fewer learned encoder layers. On Kvasir-SEG, the mean Dice score and IoU are just

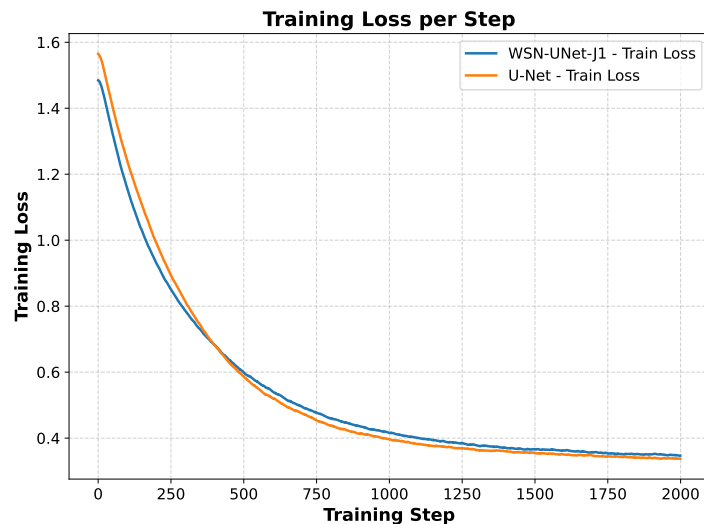


Figure 4.1: Training loss convergence for WSN-UNet-J1 and U-Net when learning on the DRIVE dataset. WSN-UNet-J1 initially converges faster for approximately the first 20 (out of 100) epochs. Loss curves are smoothed using a time-weighted exponential moving average.

0.005 and 0.004 points lower respectively. Despite this, WSN-UNet-J1 slightly outperforms U-Net in precision by 0.007 points, indicating a lower false positive rate in polyp segmentation.

While overall performance is marginally lower, this is unsurprising given the fixed filters of the scattering transform, which don't have the capacity to extract task-specific features. However, WSN-UNet-J1 demonstrates faster convergence during training in the low-data regime of DRIVE. As shown in Figure 4.1, WSN-UNet-J1 achieves lower training loss than U-Net throughout the first 20 epochs. This can be attributed to the in-built translation invariance and stability to deformations of the scattering transform, which aids the remaining learned layers when little data has been seen.

Ultimately, while U-Net achieves slightly better generalisation on the test set due to its greater depth and capacity to learn, WSN-UNet-J1 offers competitive performance with faster initial convergence. This suggests that this hybrid architecture may be a viable alternative in low-data regimes, enabling data-efficient learning and comparable generalisation.

4.1.2 WSN-UNet-J2

The performance of WSN-UNet-J2, which replaces the first four convolutional layers of U-Net with a wavelet scattering network of scale $J = 2$, is shown in Table 4.2. As seen with WSN-UNet-J1, the results show that WSN-UNet-J2 performs competitively relative to U-Net on both datasets, despite replacing half of the encoder with the non-learned scattering network. On Kvasir-SEG, the mean Dice score and IoU are 0.031 and 0.032 lower than U-Net, respectively. However, WSN-UNet-J2 achieves a 0.006 improvement in precision, indicating a better ability to avoid false positives. There is a comparable drop in performance for the smaller DRIVE dataset.

The training convergence of WSN-UNet-J2 is more similar to U-Net than WSN-UNet-J1, with both models showing similar convergence in the first 15 epochs, as depicted in Figure 4.2. Interestingly, within the first 10 epochs, WSN-UNet-J2 converges slightly faster than U-Net,

Table 4.2: Test metrics for WSN-UNet-J2 compared to U-Net on Kvasir-SEG and DRIVE.

Dataset	Model	Dice	IoU	Precision	Recall
Kvasir-SEG	U-Net	0.843 ± 0.0142	0.765 ± 0.0162	0.854 ± 0.0288	0.886 ± 0.0238
	WSN-UNet-J2	0.812 ± 0.0179	0.733 ± 0.0182	0.860 ± 0.0334	0.834 ± 0.0398
DRIVE	U-Net	0.786 ± 0.0135	0.649 ± 0.0168	0.845 ± 0.0196	0.745 ± 0.0329
	WSN-UNet-J2	0.763 ± 0.0190	0.620 ± 0.0230	0.844 ± 0.0387	0.712 ± 0.0477

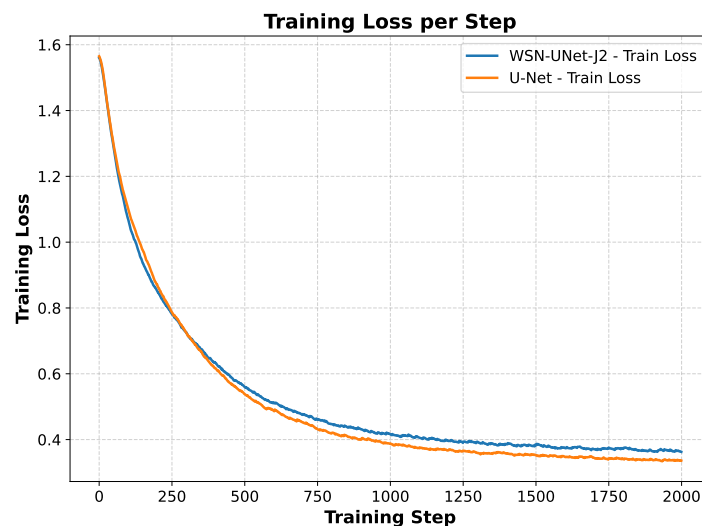


Figure 4.2: Training loss convergence for WSN-UNet-J2 and U-Net when learning on the DRIVE dataset. WSN-UNet-J2 initially converges at a similar rate for approximately the first 15 (out of 100) epochs, but U-Net maintains a lower loss thereafter. Loss curves are smoothed using a time-weighted exponential moving average.

but the two model's loss curves match by the 15th epoch. After this point, U-Net has a consistently lower training loss. This demonstrates that the wavelet scattering network can effectively replace the first half of the U-Net encoder during the early stages of training, especially in low-data regimes. However, the reduced depth of the learned portion limits the ultimate generalisation of the model, resulting in slightly lower, but competitive, performance metrics.

4.1.3 WSN-UNet-J3

WSN-UNet-J3 replaces the first six convolutional layers of the U-Net encoder with a wavelet scattering network of scale $J = 3$. Its performance relative to the U-Net baseline is presented in Table 4.3. The training loss curve on the DRIVE dataset (Figure 4.3) shows that WSN-UNet-J3 follows a similar convergence trajectory to U-Net for the first 10 epochs, after which its loss remains consistently higher. Compared to the previous two hybrid models, its convergence is slightly worse, indicating this depth of replacement may be more limiting to the network when capturing the features required for accurate segmentation for the final 90% of training. On Kvasir-SEG, we observe a decrease of 0.08 in mean Dice score and 0.09 in IoU. Precision, however, is less affected, dropping by only 0.03, again suggesting a stable ability to avoid false positives.

Table 4.3: Test metrics for WSN-UNet-J3 compared to U-Net on Kvasir-SEG and DRIVE.

Dataset	Model	Dice	IoU	Precision	Recall
Kvasir-SEG	U-Net	0.843 ± 0.0142	0.765 ± 0.0162	0.854 ± 0.0288	0.886 ± 0.0238
	WSN-UNet-J3	0.763 ± 0.0197	0.674 ± 0.0204	0.825 ± 0.0500	0.796 ± 0.0460
DRIVE	U-Net	0.786 ± 0.0135	0.649 ± 0.0168	0.845 ± 0.0196	0.745 ± 0.0329
	WSN-UNet-J3	0.763 ± 0.0169	0.619 ± 0.0211	0.823 ± 0.0607	0.728 ± 0.0663

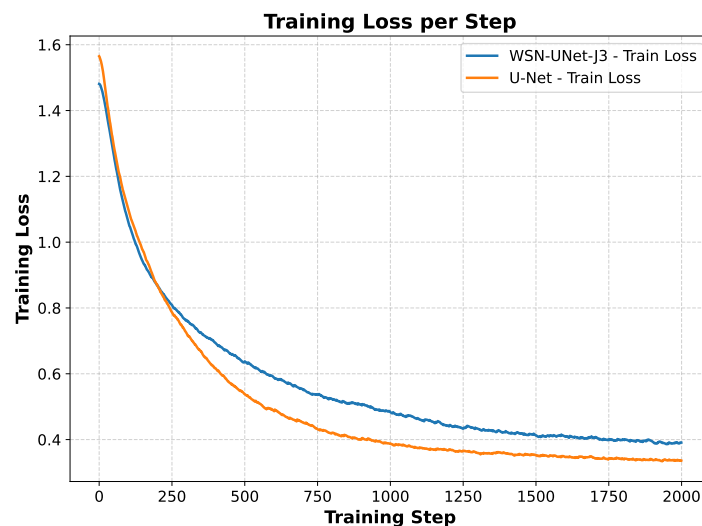


Figure 4.3: Training loss convergence for WSN-UNet-J3 and U-Net when learning on the DRIVE dataset. WSN-UNet-J3 initially converges at a similar rate for approximately the first 10 (out of 100) epochs, but U-Net maintains a lower loss thereafter. Loss curves are smoothed using a time-weighted exponential moving average.

On DRIVE, the performance drops are less pronounced with a 0.02 and 0.03 decrease in mean Dice score and IoU compared to U-Net. Interestingly, WSN-UNet-J3 achieves an identical mean Dice score to WSN-UNet-J2 and only 0.01 below WSN-UNet-J1. This suggests that the in-built translation invariance and deformation stability of scattering networks, are highly beneficial priors when training data is limited. This effect is likely pronounced due to the characteristics of the DRIVE dataset, where the scattering network's multi-scale, orientation-sensitive filters may enhance feature representation of thin, directed, vessel-like structures.

4.1.4 WSN-UNet-J4

WSN-UNet-J4 replaces the entire U-Net encoder with a wavelet scattering network of scale $J = 4$. As shown in Table 4.4, this model results in the largest drop in performance among all WSN-UNet variants. On Kvasir-SEG, the Dice score decreases by 0.17 relative to U-Net, and IoU drops by 0.19. These are accompanied by a notable decrease in precision and recall, indicating a large overall decline in segmentation quality.

This decline in performance is also reflected in the training loss curve on the DRIVE dataset (Figure 4.4), where WSN-UNet-J4 diverges from U-Net at the beginning of training, and converges significantly slower over the full 100 epochs. The scattering coefficients do not provide a sufficient latent representation to replace the learned representation from U-Net

Table 4.4: Test metrics for WSN-UNet-J4 compared to U-Net on Kvasir-SEG and DRIVE.

Dataset	Model	Dice	IoU	Precision	Recall
Kvasir-SEG	U-Net	0.843 ± 0.0142	0.765 ± 0.0162	0.854 ± 0.0288	0.886 ± 0.0238
	WSN-UNet-J4	0.677 ± 0.0211	0.577 ± 0.0197	0.773 ± 0.0500	0.7235 ± 0.0500
DRIVE	U-Net	0.786 ± 0.0135	0.649 ± 0.0168	0.845 ± 0.0196	0.745 ± 0.0329
	WSN-UNet-J4	0.721 ± 0.0246	0.566 ± 0.0284	0.768 ± 0.0818	0.704 ± 0.0736

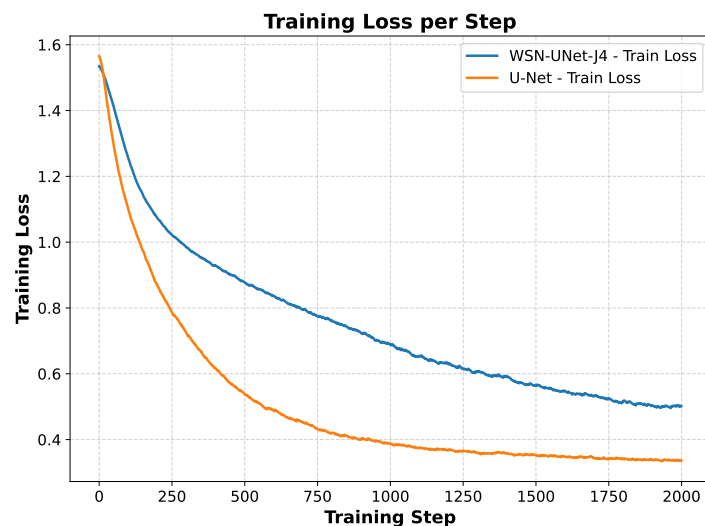


Figure 4.4: Training loss convergence for WSN-UNet-J4 and U-Net when learning on the DRIVE dataset. WSN-UNet-J4 exhibits significantly slower convergence throughout training, with a consistently higher loss than U-Net across all epochs. Loss curves are smoothed using a time-weighted exponential moving average.

encoder blocks for accurate reconstruction, leading to underfitting.

While the degradation on DRIVE is less severe, showing a 0.07 and 0.08 drop in Dice score and IoU compared to U-Net, it still reflects a substantial reduction in segmentation performance. These results continue to support the notion that the hybrid scattering framework is more advantageous when data is scarce, with less geometrical variabilities to generalise from. Nonetheless, we continue to observe an increasing drop in performance as learned encoder layers are replaced. With no learned encoder, WSN-UNet-J4 seems to have significantly less capacity to capture the complex, dataset-specific features that allow U-Net to achieve superior performance, resulting in a significant reduction in performance on both datasets.

4.1.5 Discussion

The results of all four WSN-UNet variants consistently show that replacing U-Net encoder blocks with a wavelet scattering network leads to a decrease in segmentation performance. As shown in Figure 4.5, this degradation follows an increasing non-linear trend as scattering scale J increases, which corresponds to replacing more learned encoder layers with the non-learned scattering network. The performance drop is more pronounced on the Kvasir-SEG dataset, where the U-Net encoder has the capacity to capture the extensive variabilities of the larger training set. The DRIVE dataset exhibits a smaller performance gap, which suggests that the

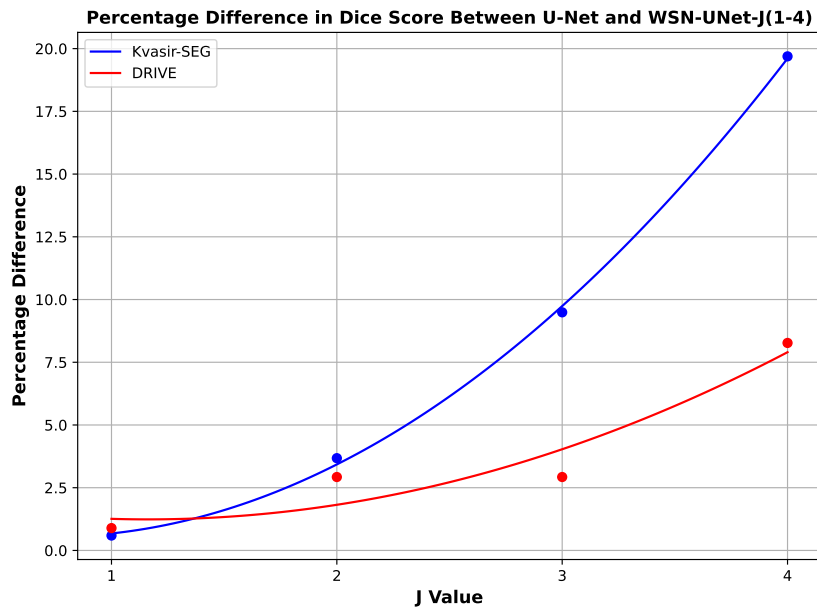


Figure 4.5: Percentage difference in Dice score between U-Net and WSN-UNet variants on Kvasir-SEG and DRIVE, plotted against scattering scale J . As more encoder layers are replaced (increasing J), the performance gap increases in a convex manner. Smaller performance differences on DRIVE suggest that the scattering network aids learning more effectively when training data is scarce and variability is limited.

scattering representation can compensate for the lack of geometrical variability, with its in-built invariance and stability. This aligns with previous work showing that scattering transforms can offer strong priors for low-data regimes (Oyallon, Belilovsky and Zagoruyko, 2017).

To understand the cause of this performance drop, we visualised feature maps from the decoders of U-Net and all WSN-UNet variants (Figure 4.6). In earlier decoder layers, better performing models retain strong low frequency features representing the polyp region, while models with fewer learned encoder layers (e.g. WSN-UNet-J4) exhibit dissimilar artifacts. Deeper into the decoder, we expect the re-introduction of fine spatial detail via the skip connections from the encoder (or scattering network). In the worst performing models, fine spatial details around polyp boundaries are smoothed or missing entirely. We attribute these effects to the lack of task-specific learning in the encoder. The non-learned scattering network cannot extract task-specific features, which impairs the decoder’s reconstruction ability.

4.2 Wavelet Enhanced U-Nets

Table 4.5 presents the performance of our proposed wavelet-enhanced variants on the Kvasir-SEG dataset. Both networks adopt the encoder from LSW-Net (Liu et al., 2022), but have different decoders: LSW-iDWT employs the inverse DWT upsampling decoder from Alijamaat, NikravanShalmani and Bayat (2021), while LSW-iDTCWT uses our inverse DTCWT-based decoder, inspired by Alijamaat, NikravanShalmani and Bayat (2021)’s design. These designs aim to replace traditional learned upsampling with interpretable wavelet-based operations.

LSW-iDTCWT achieved the highest overall performance across three metrics, obtaining a Dice score of 0.871 and an IoU of 0.802, outperforming all other published models evaluated under

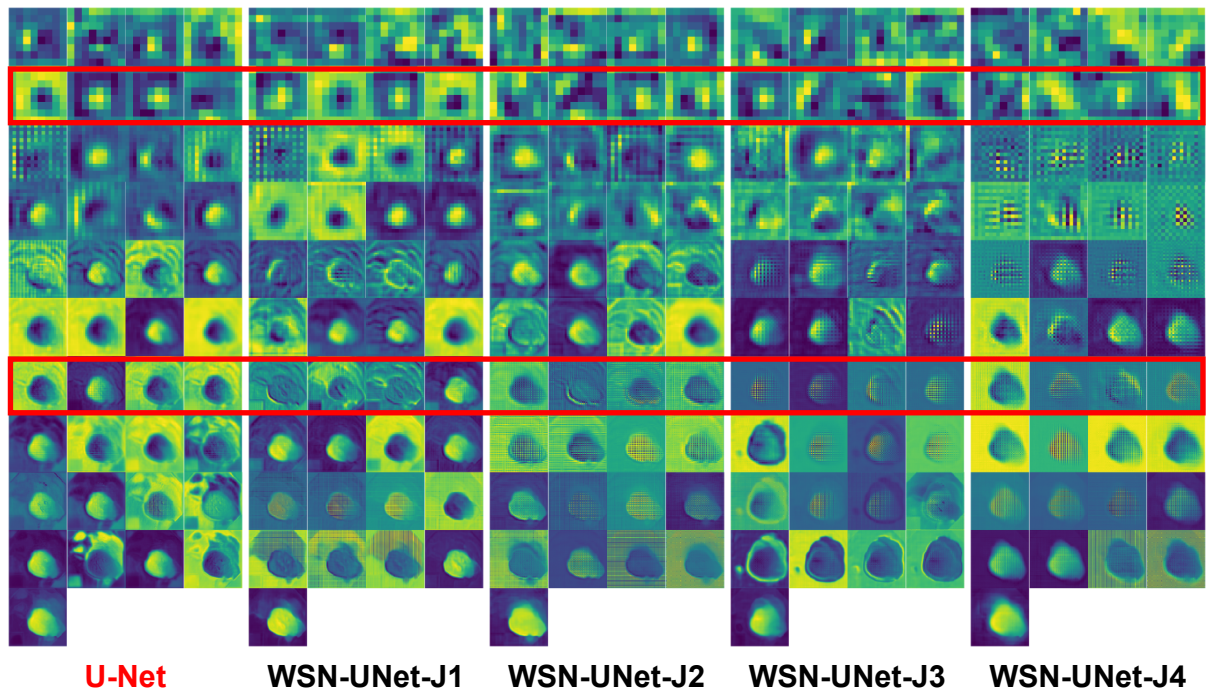


Figure 4.6: Feature map comparisons across decoder layers for U-Net and each WSN-UNet variant, ordered left to right by decreasing performance. Rows correspond to decoder layers and columns show selected feature maps at those layers. Red boxes highlight layers where the loss of coherent structured content (top) and fine spatial detail (bottom) is most evident in lower performing models.

the same training procedure. LSW-iDWT also exceeded the performance of the original LSW-Net and ranked as the second-best model overall. These results indicate that using wavelet transforms as decoder upsampling operations can improve the construction of segmentation maps. The inverse DTCWT decoder, with approximate translation invariance and directional selectivity in six orientations, may enable better reconstruction of object boundaries and fine structural details.

All three LSW-Net variants outperform all versions of WSN-UNet. This raises questions about whether the geometric priors encoded by the *DTCWT-Scat* module by Cotter (2020b) offer a more efficient alternative to hybrid scattering U-Nets. Due to its performance, LSW-iDTCWT was evaluated on DRIVE, alongside U-Net and WSN-UNet variants, to assess generalisability beyond polyp segmentation (Table 4.6). Again, it outperformed all other models across three metrics. However, it showed lower precision, suggesting overconfidence in its positive pixel predictions.

In summary, our findings suggest that integrating the inverse DTCWT into the decoder provides a more effective use of wavelet transforms in the U-Net architecture than either convolutional upsampling or inverse DWT, when paired with a DTCWT-based encoder such as that of LSW-Net.

4.3 Overall Discussion

After training implementations of existing models on Kvasir-SEG, we found that our proposed LSW-iDTCWT and LSW-iDWT architectures outperformed all published models across all performance metrics. As shown in Table 4.5, LSW-iDTCWT achieved the highest Dice score (0.871) and IoU (0.802), exceeding strong baselines such as UNet++, DUCK-Net, and the original LSW-Net, under identical training procedure, enabling fair comparison. These results highlight the effectiveness of wavelet-based downsampling and upsampling, and suggest that the interpretable, translation invariant, and directionally selective transforms should be explored beyond U-Net. While the WSN-UNet variants did not match the overall performance of most published models, they did outperform ResUNet and ResUNet++, which are intentionally shallower to highlight the benefits of residual connections. On DRIVE (Table 4.6), WSN-UNet models have a less severe drop in performance compared to U-Net, likely due to the scattering network aiding the learned portion of network when training data is scarce with few variabilities.

Figure 4.7 and Figure 4.8 show a qualitative comparison of segmentation masks produced by models over diverse test samples. In the Kvasir-SEG examples (Figure 4.7) higher performing models such as LSW-iDTCWT and U-Net++ produce segmentation masks that closely match the true mask, particularly around the polyp borders which retain appropriate structural detail. Lower performing models have generally smoother polyp borders and introduce artifacts. The DRIVE visualisations (Figure 4.8) include red bounding boxes that highlight regions of interest. These clearly depict the intricate structures that high performing models are able to identify, while masks from lower performing models increasingly lose this detail, producing broken or entirely missing blood vessels. These visual trends align with the quantitative results and continue to support the integration of complex wavelet transforms as downsampling and upsampling operators.

Our WSN-UNet variants make use of scattering coefficients computed using the Kymatio library. It is important to note that the coefficients passed through each skip connection are low-pass

Table 4.5: Test metrics for all models on Kvasir-SEG. All metrics are reported as mean $\pm$ standard deviation. Models in **red** are implementations of models proposed in existing literature, all others are ours.

Model Name	Dice Score	IoU	Precision	Recall	Params.
U-Net , 2015	0.843 $\pm$ 0.0142	0.765 $\pm$ 0.0162	0.854 $\pm$ 0.0288	0.886 $\pm$ 0.0238	31.0M
UNet++ , 2018	0.848 $\pm$ 0.0137	0.772 $\pm$ 0.0148	0.868 $\pm$ 0.0254	0.882 $\pm$ 0.0251	8.53M
ResUNet , 2018	0.795 $\pm$ 0.0179	0.703 $\pm$ 0.0200	0.828 $\pm$ 0.0441	0.838 $\pm$ 0.0384	8.22M
ResUNet++ , 2019	0.797 $\pm$ 0.0214	0.714 $\pm$ 0.0218	0.853 $\pm$ 0.0282	0.824 $\pm$ 0.0320	3.93M
DUCK-Net , 2023	0.841 $\pm$ 0.0200	0.763 $\pm$ 0.0225	0.859 $\pm$ 0.0274	0.877 $\pm$ 0.0243	40.1M
DWT U-Net , 2021	0.842 $\pm$ 0.0152	0.767 $\pm$ 0.0164	0.873 $\pm$ 0.0310	0.867 $\pm$ 0.0314	72.1M
Spectral U-Net , 2024	0.841 $\pm$ 0.0179	0.765 $\pm$ 0.0202	0.863 $\pm$ 0.0320	0.876 $\pm$ 0.0287	18.6M
LSW-Net , 2022	0.865 $\pm$ 0.0128	0.793 $\pm$ 0.0154	0.876 $\pm$ 0.0265	0.899 $\pm$ 0.0193	68.6M
LSW-iDTCWT	0.871 $\pm$ 0.0109	0.802 $\pm$ 0.0125	0.883 $\pm$ 0.0219	0.900 $\pm$ 0.0173	149M
LSW-iDWT	0.868 $\pm$ 0.0111	0.796 $\pm$ 0.0134	0.877 $\pm$ 0.0266	0.901 $\pm$ 0.0239	90.9M
WSN-UNet-J1	0.838 $\pm$ 0.0151	0.761 $\pm$ 0.0168	0.861 $\pm$ 0.0328	0.870 $\pm$ 0.0320	31.0M
WSN-UNet-J2	0.812 $\pm$ 0.0179	0.733 $\pm$ 0.0182	0.860 $\pm$ 0.0334	0.834 $\pm$ 0.0398	31.2M
WSN-UNet-J3	0.763 $\pm$ 0.0197	0.674 $\pm$ 0.0204	0.825 $\pm$ 0.0500	0.796 $\pm$ 0.0460	32.5M
WSN-UNet-J4	0.677 $\pm$ 0.0211	0.577 $\pm$ 0.0197	0.773 $\pm$ 0.0500	0.7235 $\pm$ 0.0500	37.2M

Table 4.6: Test metrics for models on DRIVE. All metrics are reported as mean $\pm$ standard deviation. Models in **red** are implementations of models proposed in existing literature, all others are ours.

Model Name	Dice Score	IoU	Precision	Recall	Params.
U-Net , 2015	0.786 $\pm$ 0.0135	0.649 $\pm$ 0.0168	0.845 $\pm$ 0.0196	0.745 $\pm$ 0.0329	31.0M
DTCWT-UNet	0.791 $\pm$ 0.0087	0.655 $\pm$ 0.0111	0.812 $\pm$ 0.0243	0.780 $\pm$ 0.0242	149M
WSN-UNet-J1	0.779 $\pm$ 0.0188	0.642 $\pm$ 0.0223	0.823 $\pm$ 0.0505	0.758 $\pm$ 0.0417	31.0M
WSN-UNet-J2	0.763 $\pm$ 0.0190	0.620 $\pm$ 0.0230	0.844 $\pm$ 0.0387	0.712 $\pm$ 0.0477	31.2M
WSN-UNet-J3	0.763 $\pm$ 0.0169	0.619 $\pm$ 0.0211	0.823 $\pm$ 0.0607	0.728 $\pm$ 0.0663	32.5M
WSN-UNet-J4	0.721 $\pm$ 0.0246	0.566 $\pm$ 0.0284	0.768 $\pm$ 0.0818	0.704 $\pm$ 0.0736	37.2M

filtered during the scattering transform. This filtering introduces local translation invariance by smoothing the output, which results in the loss of high frequency content, necessary for accurate segmentation. In our implementations, we restore spatial dimensions by then applying additional up-convolutional layers to these averaged coefficients before insertion into the decoder path. However, a better approach would be to extract and skip the non-low-pass (unaveraged) coefficients directly, preserving more high-frequency content. This would reduce the number of up-convolutions required per skip connection and could enhance the decoder’s ability to reconstruct sharp object boundaries and fine structures. Due to the complexity of modifying the Kymatio library’s internals within the time constraints of this project, this optimisation was not made and will be left for future work.

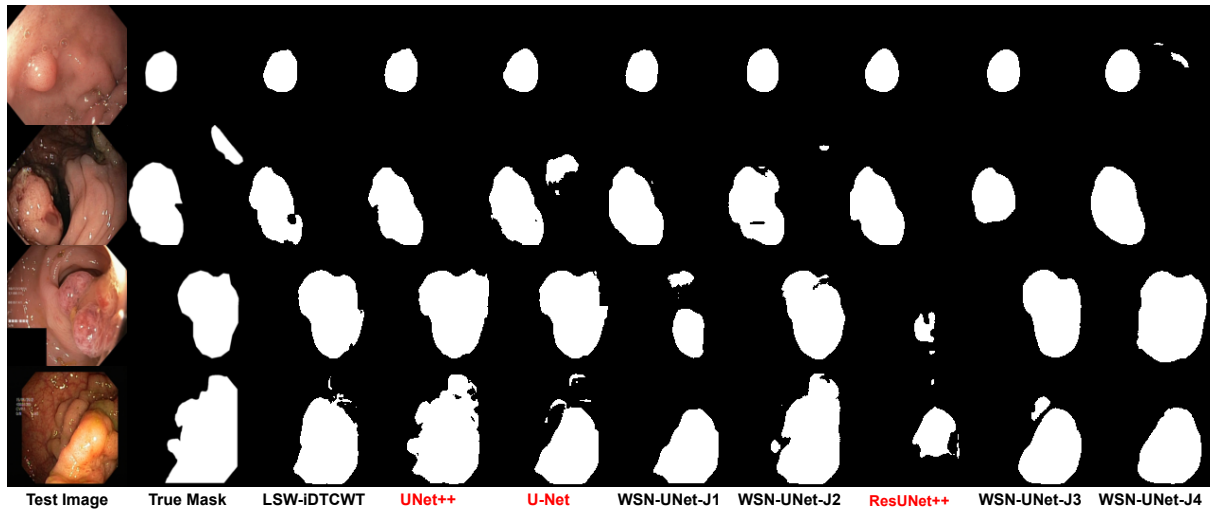


Figure 4.7: Qualitative comparison of segmentation masks on the Kvasir-SEG dataset. Each row shows a test sample. From left to right: input image, ground truth mask, and predicted masks from selected models in descending order of performance. Higher performing models capture sharper polyp boundaries, while lower performing models exhibit artefacts and smoother borders.

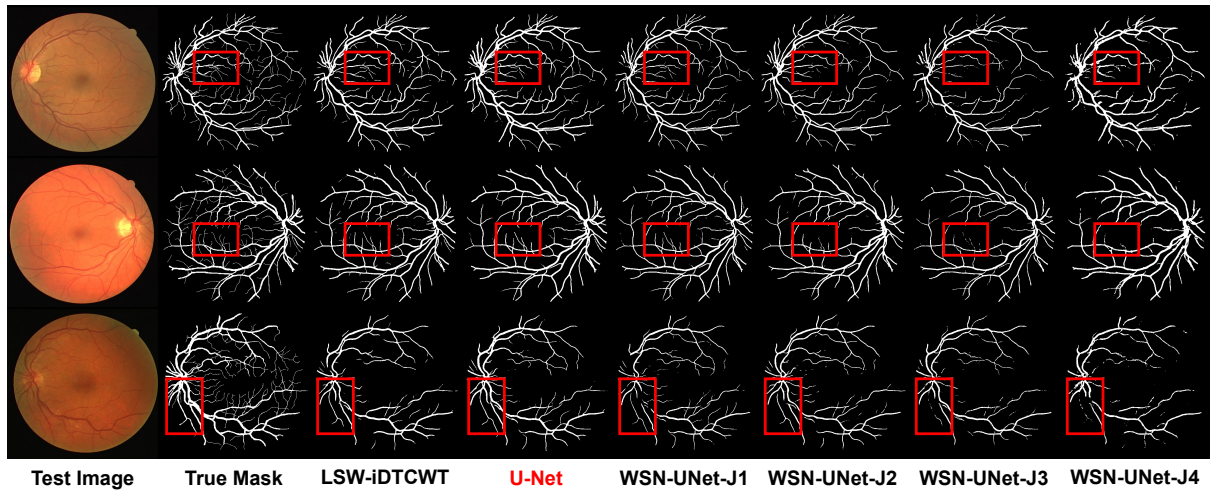


Figure 4.8: Qualitative comparison of segmentation masks on the DRIVE dataset. Each row shows a test sample. From left to right: input image, ground truth mask, and predicted masks from each model in descending order of performance. Red boxes highlight areas of interest where fine spatial detail is better preserved in high performing models and progressively lost in lower performing ones.

Chapter 5

Conclusions

This dissertation explored the integration of wavelet transforms into the U-Net architecture for medical semantic segmentation, focusing on two distinct strategies:

1. Replacing U-Net encoder layers with a fixed wavelet scattering network
2. Substituting traditional downsampling (max-pooling) and upsampling (up-convolutions) operations with both complex and real-valued wavelet transforms.

The project was guided by three core research questions: understanding the effects of varying the depth of encoder replacement, data efficiency, and comparative effectiveness of wavelet-based downsampling and upsampling.

To address the first two questions, four hybrid models (WSN-UNet-J(1-4)) were proposed, each replacing a progressively greater portion of the U-Net encoder with a wavelet scattering network. We analysed how the extent of replacement affects segmentation performance and training dynamics. On the larger Kvasir-SEG dataset, results showed a clear increasing degradation in segmentation quality as more encoder layers were replaced. However, on the smaller DRIVE dataset, this performance drop was less pronounced, and the early convergence of WSN-UNet-J1 and WSN-UNet-J2 demonstrated improved data efficiency. This suggests that the in-built invariance and deformation stability of scattering transforms can be especially advantageous when training data is scarce.

We designed two wavelet-enhanced models, LSW-iDWT and LSW-iDTCWT, which combined the encoder of LSW-Net (Liu et al., 2022) with wavelet-based upsampling decoders. These models demonstrated that wavelet transforms can outperform traditional pooling and upsampling layers. LSW-iDTCWT achieved the best performance of all tested models on Kvasir-SEG, outperforming established baselines such as U-Net++, DUCK-Net and the original LSW-Net, and also generalised well to the smaller DRIVE dataset. The DTCWT, with its approximate translation invariance and directional selectivity, was found to preserve more fine spatial detail than the real-valued DWT. Both models were found to be particularly effective for accurate segmentation after visual qualitative examination and analysis of quantitative data.

A limitation identified in the WSN-UNet variants relates to the use of low-pass filtered scattering coefficients for skip connections, which led to a loss of high frequency content. To compensate, additional up-convolutional layers were introduced. A cleaner solution would involve modifying the Kymatio library to extract non-averaged coefficients, introducing finer detail into the

decoder and reducing reliance on learned upsampling. This provides a clear starting point for future work.

Additionally, while this project focused on U-Net architecture, future experiments could investigate the integration of wavelet transforms into better performing models such as UNet++, DUCK-Net, or attention-based models. Additionally, extending evaluation to multi-label tasks and larger datasets like COCO would offer further insight into the generalisability and robustness of proposed models.

In summary, this project demonstrated that both scattering and wavelet transforms can offer effective and interpretable alternatives to conventional CNN architectures. While fixed wavelet scattering hybrid encoders have benefits in low-data regimes, the most promising improvements were achieved through wavelet-based upsampling and downsampling, especially when using the DTCWT. These findings support the broader notion of integrating signal processing techniques into deep learning architectures for improved signal representation and segmentation performance.

Bibliography

- Agnes, S.A., Solomon, A.A. and Karthick, K., 2024. Wavelet u-net++ for accurate lung nodule segmentation in ct scans: Improving early detection and diagnosis of lung cancer. *Biomedical signal processing and control*, 87, p.105509.
- Alijamaat, A., NikravanShalmani, A. and Bayat, P., 2021. Multiple sclerosis lesion segmentation from brain mri using u-net based on wavelet pooling. *International journal of computer assisted radiology and surgery*, 16(9), pp.1459–1467.
- Andreux, M., Angles, T., Exarchakis, G., Leonarduzzi, R., Rochette, G., Thiry, L., Zarka, J., Mallat, S., AndÅ©n, J., Belilovsky, E., Bruna, J., Lostanlen, V., Chaudhary, M., Hirn, M.J., Oyallon, E., Zhang, S., Cella, C. and Eickenberg, M., 2020. Kymatio: Scattering transforms in python. *Journal of machine learning research* [Online], 21(60), pp.1–6. Available from: <http://jmlr.org/papers/v21/19-047.html>.
- Bergstra, J. and Bengio, Y., 2012. Random search for hyper-parameter optimization. *The journal of machine learning research*, 13(1), pp.281–305.
- Bruna, J. and Mallat, S., 2013. Invariant scattering convolution networks. *IEEE transactions on pattern analysis and machine intelligence*, 35(8), pp.1872–1886.
- Chen, J., Lu, Y., Yu, Q., Luo, X., Adeli, E., Wang, Y., Lu, L., Yuille, A.L. and Zhou, Y., 2021. Transunet: Transformers make strong encoders for medical image segmentation. *arxiv preprint arxiv:2102.04306*.
- Chen, L.C., Papandreou, G., Schroff, F. and Adam, H., 2019. Rethinking atrous convolution for semantic image segmentation. *arxiv 2017. arxiv preprint arxiv:1706.05587*, 2, p.1.
- Cooley, J.W. and Tukey, J.W., 1965. An algorithm for the machine calculation of complex fourier series. *Mathematics of computation*, 19(90), pp.297–301.
- Cotter, F., 2020a. *pytorch_wavelets*. https://github.com/fbcotter/pytorch_wavelets.
- Cotter, F., 2020b. *Uses of complex wavelets in deep convolutional neural networks*. Ph.D. thesis.
- Dong, B., Wang, W., Fan, D.P., Li, J., Fu, H. and Shao, L., 2021. Polyp-pvt: Polyp segmentation with pyramid vision transformers. *arxiv preprint arxiv:2108.06932*.
- Dumitru, R.G., Peteleaza, D. and Craciun, C., 2023. Using duck-net for polyp image segmentation. *Scientific reports*, 13(1), p.9803.
- He, K., Zhang, X., Ren, S. and Sun, J., 2016. Deep residual learning for image recognition. *Proceedings of the IEEE conference on computer vision and pattern recognition*. pp.770–778.

- Head, T., Cherti, M., Pedregosa, F., Zhdanov, M., Louppe, G., Raffel, C., Mueller, A., Fauchere, N., McInnes, L. and Grisel, O., 2018. Scikit-optimize: Sequential model-based optimization with scikit-learn. 2018.
- Huh, M., Agrawal, P. and Efros, A.A., 2016. What makes imagenet good for transfer learning? *arxiv preprint arxiv:1608.08614*.
- Ioffe, S. and Szegedy, C., 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *International conference on machine learning*. pmlr, pp.448–456.
- Jha, D., Smedsrud, P.H., Riegler, M.A., Halvorsen, P., Lange, T. de, Johansen, D. and Johansen, H.D., 2020. Kvasir-seg: A segmented polyp dataset. *International conference on multimedia modeling*. Springer, pp.451–462.
- Jha, D., Smedsrud, P.H., Riegler, M.A., Johansen, D., De Lange, T., Halvorsen, P. and Johansen, H.D., 2019. Resunet++: An advanced architecture for medical image segmentation. *2019 IEEE International Symposium on Multimedia (ISM)*. IEEE, pp.225–2255.
- Kingsbury, N.G., 1998. The dual-tree complex wavelet transform: a new technique for shift invariance and directional filters. *IEEE digital signal processing workshop*. Citeseer, vol. 86, pp.120–131.
- Krizhevsky, A., Sutskever, I. and Hinton, G.E., 2012. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25.
- LeCun, Y., Boser, B., Denker, J.S., Henderson, D., Howard, R.E., Hubbard, W. and Jackel, L.D., 1989. Backpropagation applied to handwritten zip code recognition. *Neural computation* [Online], 1(4), pp.541–551. Available from: <https://doi.org/10.1162/neco.1989.1.4.541>.
- Li, Z., Liu, F., Yang, W., Peng, S. and Zhou, J., 2021. A survey of convolutional neural networks: analysis, applications, and prospects. *IEEE transactions on neural networks and learning systems*, 33(12), pp.6999–7019.
- Liu, R., Nan, H., Zou, Y., Xie, T. and Ye, Z., 2022. Lsw-net: A learning scattering wavelet network for brain tumor and retinal image segmentation. *Electronics*, 11(16), p.2616.
- Long, J., Shelhamer, E. and Darrell, T., 2015. Fully convolutional networks for semantic segmentation. *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*.
- Mallat, S., 1999. A wavelet tour of signal processing.
- Mallat, S., 2012. Group invariant scattering. *Communications on pure and applied mathematics*, 65(10), pp.1331–1398.
- Mallat, S.G., 1989. A theory for multiresolution signal decomposition: the wavelet representation. *IEEE transactions on pattern analysis and machine intelligence*, 11(7), pp.674–693.
- Mohammed, A., 2022. Resattunet: detecting marine debris using an attention activated residual unet. *arxiv preprint arxiv:2210.08506*.
- Oyallon, E., Belilovsky, E. and Zagoruyko, S., 2017. Scaling the scattering transform: Deep

- hybrid networks. *Proceedings of the ieee international conference on computer vision*. pp.5618–5627.
- Oyallon, E. and Mallat, S., 2015. Deep roto-translation scattering for object classification. *Proceedings of the ieee conference on computer vision and pattern recognition*. pp.2865–2873.
- Paszke, A., 2019. Pytorch: An imperative style, high-performance deep learning library. *arxiv preprint arxiv:1912.01703*.
- Peng, Y., Sonka, M. and Chen, D.Z., 2024. Spectral u-net: Enhancing medical image segmentation via spectral decomposition. *arxiv preprint arxiv:2409.09216*.
- Renard, F., Guedria, S., Palma, N.D. and Vuillerme, N., 2020. Variability and reproducibility in deep learning for medical image segmentation. *Scientific reports*, 10(1), p.13724.
- Ronneberger, O., Fischer, P. and Brox, T., 2015. U-net: Convolutional networks for biomedical image segmentation. In: N. Navab, J. Hornegger, W.M. Wells and A.F. Frangi, eds. *Medical image computing and computer-assisted intervention – miccai 2015*. Cham: Springer International Publishing, pp.234–241.
- Sanderson, E. and Matuszewski, B.J., 2022. Fcn-transformer feature fusion for polyp segmentation. *Annual conference on medical image understanding and analysis*. Springer, pp.892–907.
- Sifre, L. and Mallat, S., 2013. Rotation, scaling and deformation invariant scattering for texture discrimination. *Proceedings of the ieee conference on computer vision and pattern recognition*. pp.1233–1240.
- Simonyan, K., 2014. Very deep convolutional networks for large-scale image recognition. *arxiv preprint arxiv:1409.1556*.
- Snoek, J., Larochelle, H. and Adams, R.P., 2012. Practical bayesian optimization of machine learning algorithms. *Advances in neural information processing systems*, 25.
- Staal, J., Abràmoff, M.D., Niemeijer, M., Viergever, M.A. and Van Ginneken, B., 2004. Ridge-based vessel segmentation in color images of the retina. *IEEE transactions on medical imaging*, 23(4), pp.501–509.
- Van Rijn, J.C., Reitsma, J.B., Stoker, J., Bossuyt, P.M., Van Deventer, S.J. and Dekker, E., 2006. Polyp miss rate determined by tandem colonoscopy: a systematic review. *Official journal of the american college of gastroenterology| acg*, 101(2), pp.343–350.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I., 2017. Attention is all you need. *Advances in neural information processing systems*, 30.
- Waldspurger, I., 2017. Wavelet transform modulus: phase retrieval and scattering. *Journées équations aux dérivées partielles*, pp.1–10.
- Woo, S., Park, J., Lee, J.Y. and Kweon, I.S., 2018. Cbam: Convolutional block attention module. *Proceedings of the european conference on computer vision (eccv)*. pp.3–19.
- Yang, L. and Shami, A., 2020. On hyperparameter optimization of machine learning algorithms: Theory and practice. *Neurocomputing*, 415, pp.295–316.

- Zhang, Z., Liu, Q. and Wang, Y., 2018. Road extraction by deep residual u-net. *IEEE geoscience and remote sensing letters*, 15(5), pp.749–753.
- Zhou, Z., Rahman Siddiquee, M.M., Tajbakhsh, N. and Liang, J., 2018. Unet++: A nested u-net architecture for medical image segmentation. *Deep learning in medical image analysis and multimodal learning for clinical decision support: 4th international workshop, dlmia 2018, and 8th international workshop, ml-cds 2018, held in conjunction with miccai 2018, granada, spain, september 20, 2018, proceedings 4*. Springer, pp.3–11.

Appendix A

Optimal Hyperparameters

This appendix details the optimal hyperparameter values identified for each model following Bayesian hyperparameter tuning. Values were tuned on the validation set of Kvasir-SEG.

Table A.1: Optimal hyperparameter values for each model, selected using Bayesian optimisation. Models in **red** are implementations of proposed models in existing literature, all others are ours.

Model Name	Learning Rate	Weight Decay	Gradient Clipping
U-Net 2015	1.8662e-4	1.171e-6	1.756
UNet++ 2018	3.3740e-4	1.0000e-2	2.000
ResUNet 2018	1.2949e-4	1.0000e-2	2.000
ResUNet++ 2019	4.6134e-4	8.5970e-4	2.000
DUCK-Net 2023	5.1976e-4	1.7859e-8	1.996
DWT U-Net 2021	1.5301e-4	1.0000e-2	0.500
Spectral U-Net 2024	1.2247e-4	2.8040e-7	1.904
LSW-Net 2022	1.5921e-4	1.0000e-2	0.500
LSW-iDTCWT	7.9818e-5	5.4899e-3	0.504
WSN-UNet-J1	1.4046e-4	6.3326e-4	2.000
WSN-UNet-J2	1.6724e-4	8.3200e-6	0.500
WSN-UNet-J3	1.2609e-4	1.0000e-8	0.500
WSN-UNet-J4	7.4170e-5	9.5696e-3	1.69

Appendix B

Bayesian Credible Intervals of Metrics

This appendix presents the 95% Bayesian credible intervals for each evaluation metric of all models trained on the Kvasir-SEG dataset. These intervals were computed at the end of each training run and represent the range within which the true metric value is likely to lie, with 95% posterior probability.

Table B.1: Bayesian 95% credible intervals for metrics of all models on Kvasir-SEG. Intervals are reported as (lower bound, upper bound) and rounded to 3.s.f. Models in **red** are implementations of models proposed in existing literature, all others are ours.

Model Name	Dice Score	IoU	Precision	Recall
U-Net , 2015	(0.836, 0.850)	(0.756, 0.773)	(0.846, 0.860)	(0.880, 0.893)
UNet++ , 2018	(0.841, 0.855)	(0.764, 0.780)	(0.861, 0.874)	(0.875, 0.888)
ResUNet , 2018	(0.787, 0.803)	(0.693, 0.711)	(0.821, 0.835)	(0.830, 0.845)
ResUNet++ , 2019	(0.789, 0.805)	(0.705, 0.723)	(0.846, 0.860)	(0.816, 0.831)
DUCK-Net , 2023	(0.833, 0.849)	(0.753, 0.772)	(0.851, 0.867)	(0.870, 0.884)
DWT U-Net , 2021	(0.834, 0.848)	(0.759, 0.775)	(0.867, 0.880)	(0.860, 0.873)
Spectral U-Net , 2024	(0.834, 0.848)	(0.757, 0.773)	(0.856, 0.869)	(0.870, 0.883)
LSW-Net , 2022	(0.859, 0.872)	(0.785, 0.800)	(0.870, 0.883)	(0.893, 0.905)
LSW-iDTCWT	(0.864, 0.878)	(0.794, 0.809)	(0.876, 0.889)	(0.894, 0.906)
WSN-UNet-J1	(0.830, 0.845)	(0.752, 0.769)	(0.854, 0.868)	(0.863, 0.876)
WSN-UNet-J2	(0.804, 0.820)	(0.724, 0.741)	(0.853, 0.867)	(0.827, 0.841)
WSN-UNet-J3	(0.755, 0.772)	(0.664, 0.683)	(0.818, 0.833)	(0.788, 0.803)
WSN-UNet-J4	(0.668, 0.686)	(0.567, 0.587)	(0.764, 0.781)	(0.715, 0.732)

Appendix C

Enhancing Decoder Input with Raw RGB Features

Skip connections in U-Net architectures typically transfer multi-scale feature maps from the encoder to the decoder, helping preserve spatial detail. To explore whether the raw input image contains useful complementary information not captured in deeper features, we experimented with an additional skip connection from the input RGB channels directly to the first decoder block.

The comparison in Table C.1 shows a marginal improvement in segmentation performance on the Kvasir-SEG dataset when this modification is applied. The result suggests that incorporating low-level input information may provide a small benefit in some cases.

Table C.1: Comparison of Kvasir-SEG segmentation performance (Dice score) with and without concatenating the input RGB channels to the first decoder skip connection in U-Net.

Model	Dice Score
Standard U-Net	0.842 $\pm$ 0.0160
U-Net + Input-to-Decoder Skip	0.843 $\pm$ 0.0142

Appendix D

Evaluating the Role of Intermediate Skip Connections

A key feature of the U-Net architecture is the use of skip connections, which introduce spatial detail from encoder layers directly into their corresponding decoder layers. These connections preserve localisation and improve reconstruction.

In the context of wavelet scattering hybrid U-Nets, removing encoder blocks as done in WSN-UNet-J2 through WSN-UNet-J4, results in the loss of these beneficial skip connections. To address this, we proposed restoring the intermediate skip connections using scattering coefficients, grouped by scattering order as detailed in subsection 3.2.1, page 21. These order-based groupings are chosen to correspond approximately to the level of abstraction typically found at the respective depths in the encoder.

To validate this design, we compare each hybrid model against a counterpart that instead concatenates all scattering coefficients only at the final decoder block. These baseline variants are denoted by the suffix NSC (No Skip Connections).

Results in Table D.1 clearly demonstrate that omitting intermediate skip connections degrades performance across all key metrics, supporting the necessity of order-aligned skip restoration.

Table D.1: Test metrics comparing WSN-UNet-J2 through J4 with and without intermediate skip connections on Kvasir-SEG.

Model	Dice	IoU	Precision	Recall
WSN-UNet-J2	0.812 $\pm$ 0.0179	0.733 $\pm$ 0.0182	0.860 $\pm$ 0.0334	0.834 $\pm$ 0.0398
WSN-UNet-J2-NSC	0.801 $\pm$ 0.0269	0.712 $\pm$ 0.0264	0.843 $\pm$ 0.0252	0.830 $\pm$ 0.0471
WSN-UNet-J3	0.763 $\pm$ 0.0197	0.674 $\pm$ 0.0204	0.825 $\pm$ 0.0500	0.796 $\pm$ 0.0460
WSN-UNet-J3-NSC	0.747 $\pm$ 0.0296	0.658 $\pm$ 0.0263	0.833 $\pm$ 0.0371	0.775 $\pm$ 0.0489
WSN-UNet-J4	0.677 $\pm$ 0.0211	0.577 $\pm$ 0.0197	0.773 $\pm$ 0.0500	0.724 $\pm$ 0.0500
WSN-UNet-J4-NSC	0.675 $\pm$ 0.0127	0.576 $\pm$ 0.0133	0.787 $\pm$ 0.0425	0.707 $\pm$ 0.0317

Appendix E

Data Preparation and Training Procedure Differences for DRIVE

Kvasir-SEG Dataset: the primary dataset used in our evaluation of the proposed models. The dataset was processed as follows:

- Epochs: 50
- Batch Size: 8
- Image Resize: The images were resized to a resolution of (128, 128) pixels.
- Data Split: The dataset was divided into 80% training, 10% validation, and 10% testing.

DRIVE Dataset: was used for evaluation to assess model generalisability and model performance with limited training data. The following preparation was applied to the DRIVE dataset:

- Epochs: 100
- Batch Size: 1
- Image Resize: Images were used at their original resolution of (564, 584) (width, height)
- Data Split: The dataset is split into 50% training and 50% testing of 20 images each.

Training Procedure: Both datasets followed the same data augmentation procedures and hyperparameter search strategies.

Appendix F

WaveletUp Module: Inverse DTCWT Decoder Block

The following code shows the PyTorch implementation of the WaveletUp module used in the LSW-iDTCWT decoder. This module performs upsampling using the inverse Dual-Tree Complex Wavelet Transform, and concatenates a skip connection as part of the decoding process.

Listing F.1: WaveletUp module: DTCWT-based upsampling block used in the LSW-iDTCWT decoder

```
class WaveletUp(nn.Module):
    """Upscaling using inverse DTCWT with skip connection"""
    def __init__(self, in_channels, out_channels, n_final_skip=0):
        super().__init__()
        # Firstly a single conv to 6.5x number of features (this
        # guarantees divisible by 13)
        self.single_conv = SingleConv(in_channels, int(6.5 *
            in_channels))

        # Then upsample 1/13th for y1
        self.up = nn.ConvTranspose2d(in_channels // 2, in_channels
            // 2, kernel_size=2, stride=2)

        # Followed by the idwt, which will 1/13 the number of
        # features.
        self.idwt = DTCWTInverse().cuda()

        # Finally, we have ordinary double convolution to expected
        # number of out channels
        expected_in_channels = (in_channels // 2 + n_final_skip) if
            n_final_skip > 0 else in_channels
        self.double_conv = DoubleConv(expected_in_channels,
            out_channels)

    def forward(self, x, skip_connection):
        x = self.single_conv(x)
```

```
batch, channels, height, width = x.shape
num_base_channels = channels // 13
Yl, Yh = torch.split(x, [num_base_channels, 12 *
    num_base_channels], dim=1)

Yl = self.up(Yl) # upsample yl features

# Yh shape: (N, C, O(rientations), H, W, I(real or
    imaginary))
Yh = rearrange(Yh, 'b (c o i) h w -> b c o h w i', i=2, o=6)

# Perform inverse DTCWT
x = self.idwt((Yl, [Yh]))

return self.double_conv(torch.cat([skip_connection, x],
    dim=1))
```

Appendix G

Code Repository and Datasets

Code Repository: <https://github.com/jamesmedley/Individual-Project>

Kvasir-SEG polyp segmentation dataset: <https://datasets.simula.no/kvasir-seg/>

DRIVE: Digital Retinal Images for Vessel Extraction dataset: <https://drive.grand-challenge.org/>